



WICHITA STATE
UNIVERSITY

UNIVERSITY LIBRARIES

**A brain-inspired framework for
evolutionary artificial general intelligence**

Item Type	Dissertation
Authors	Nadji-Tehrani, Mohammad
Publisher	Wichita State University
Rights	Copyright 2020 by Mohammad Nadji-Tehrani All Rights Reserved
Download date	2026-05-14 11:35:37
Link to Item	https://soar.wichita.edu/handle/10057/18815

A BRAIN-INSPIRED FRAMEWORK FOR
EVOLUTIONARY ARTIFICIAL GENERAL INTELLIGENCE

A Dissertation by

Mohammad Nadji-Tehrani

Master of Science, Wichita State University, 2005

Bachelor of Science, Qazvin Islamic Azad University, 2002

Submitted to the Department of Electrical Engineering and Computer Science
and the faculty of the Graduate School of
Wichita State University
in partial fulfillment of
the requirements for the degree of
Doctor of Philosophy

May 2020

© Copyright 2020 by Mohammad Nadji-Tehrani
All Rights Reserved

A BRAIN-INSPIRED FRAMEWORK FOR
EVOLUTIONARY ARTIFICIAL GENERAL INTELLIGENCE

The following faculty members have examined the final copy of this dissertation for form and content, and recommend that it be accepted in partial fulfillment of the requirement for the degree of Doctor of Philosophy, with a major in Electrical Engineering and Computer Science.

Ali Eslami, Chair

Preethika Kumar, Member

Kaushik Sinha, Member

Sergio Salinas, Member

Barbara Smith, Outside Member

Accepted for the College of Engineering

Dennis Livesay, Dean

Accepted for the Graduate School

Coleen Pugh, Dean

DEDICATION

To my loved ones

Through the power of the mind, a new
intelligent species will soon come into existence.

ACKNOWLEDGEMENTS

This research would not have been possible without the help and support of many who I like to extend my appreciation to. I want to thank my lovely family, especially my parents, for all the sacrifices they made to lay the foundation for my education along with their continuous encouragement and support. I would also like to thank my research advisor, Dr. Ali Eslami, for his invaluable guidance throughout the research. I extend my gratitude to my committee members, Dr. Preethika Kumar, Dr. Barbara Smith, Dr. Sergio Salinas, and especially Dr. Kaushik Sinha, for his valuable feedback that has influenced the quality of this research. I want to thank Kristie Bixby for her extensive editorial assistance on my work. Last but not least, I would like to thank NetApp Inc. and my colleagues for their support, enabling me to go through this journey. . .

ABSTRACT

From the medical field to agriculture, from energy to transportation, every industry is going through a revolution by embracing artificial intelligence (AI); nevertheless, AI is still in its infancy when compared to the cognitive abilities of a biological brain. To bridge this gap, inspired by the evolution of the human brain, this research demonstrates a novel method and framework to synthesize an artificial brain with cognitive abilities by taking advantage of the same process responsible for the growth of the biological brain called “neuroembryogenesis.” This framework shares some of the key behavioral aspects of the biological brain such as spiking neurons, neuroplasticity, neuronal pruning, and excitatory and inhibitory interactions between neurons, together making it capable of learning and memorizing. One of the highlights of the proposed design is its potential to incrementally improve itself over generations based on system performance, using genetic algorithms. Achieving artificial general intelligence (AGI) is not an easy task; yet, the methodology and approach discussed in this research could pave the way and make it achievable. A proof of concept at the end of this writing demonstrates how a simplified implementation of the human visual cortex using the proposed framework is capable of character recognition. The entire codebase has been open sourced and is accessible at www.FEAGI.org.

TABLE OF CONTENTS

Chapter	Page
1. INTRODUCTION	1
1.1 Background	1
1.2 State of the Art	4
2. FRAMEWORK ARCHITECTURE	9
2.1 Framework Overview	9
2.2 Genome	11
2.3 Connectome	13
2.4 Neuroembryogenesis Unit	13
2.4.1 Cortical Area Formation	14
2.4.2 Neurogenesis	14
2.4.3 Synaptogenesis	16
2.5 Input/Output Processing Unit	17
2.5.1 Transforming an Image Using Kernel Convolution	20
2.6 Neural Processing Unit	22
2.6.1 Neuron Firing, Queues, and Bursts	22
2.6.2 Neurotransmitters	25
2.6.3 Neuroplasticity	25
2.6.4 Spiking Neuron Model	27
2.6.5 Refractory Period	29
2.6.6 Synaptic Pruning	30
3. MEMORY AND LEARNING	31
3.1 Memory Design	31
3.2 Memorization Process	31
3.3 Declarative Memory	32
3.4 Memory Recall Process	34
3.5 Associative Learning	34
4. TRAINING AND TESTING	38
4.1 Self-Learning	38
4.2 Training Methods	38
4.3 Testing	40
5. EVOLUTIONARY MODULE	42
5.1 Genome Database	42
5.2 Genetic Algorithms	44

TABLE OF CONTENTS (continued)

Chapter	Page
6. FRAMEWORK HIGHLIGHTS	46
6.1 Scalability	46
6.2 Evolvability	48
6.3 Insights	49
7. RESULTS AND DISCUSSION	52
7.1 Proof of Concept	52
7.2 Results and Discussion	56
7.2.1 Performance Improvements over Time	57
7.2.2 Comprehension in Real-Time	60
7.2.3 Framework Versatility	60
7.2.4 Brain Footprint	61
7.2.5 Impact of Exposure Time on Fitness	61
8. CONCLUSION AND FUTURE WORK	64
8.1 Conclusion	64
8.2 Future Work	64
REFERENCES	66
APPENDICES	73
A. SAMPLE CONNECTOME	74
B. SAMPLE GENOME	82

LIST OF FIGURES

Figure	Page
1.1 Diversity of approaches laying the foundation for AGI.	5
1.2 Timeline for key prenatal human brain development phases that are accounted for as part of framework design.	8
2.1 FEAGI's software ecosystem.	10
2.2 Relation between a biological gene and its counterpart in FEAGI.	12
2.3 Artificial brain formation that begins by reading genome instructions and going through multiple phases sequentially, eventually capturing neurons and synapses that are created throughout the process in an object-oriented database, or connectome, for future use as part of the artificial brain's operations.	14
2.4 Growth rules as a set of geometrical formulas defining space in which axon of source neuron can find its destination neuron, defined in Cartesian coordinates and inspired by morphology of biological neurons; source neuron's coordinates represented by $[X_1, Y_1, Z_1]$ and a potential destination neuron's coordinate represented by $[X_2, Y_2, Z_2]$: (a) growth rule "a" defines a cylindrical boundary into which the destination neuron coordinates can fall, where h is neuron's distance from the cylinder's base, h' is height and r is radius of cylinder; (b) growth rule "b" simulates cone-shaped synaptogenesis region similar to that occupied by pyramidal neuron's axons, where h is height of the cone, and r is radius of its base; (c) growth rule "c" represents spherical-shaped neuron with axon length reaching maximum r as sphere's radius.	18
2.5 Visual pathway in human brain.....	19
2.6 Flow of information through vision IPU.	20
2.7 Connectivity between cortical layers representing UTF-8 module.	20
2.8 Artificial brain built by FEAGI running on bursts, whereby list of neurons ready to fire is added to fire queue, and they all fire simultaneously when burst is triggered, a mechanism used to overcome the parallel nature of spiking neurons when implemented in environment with limitations in parallelism.	23
2.9 Sequence diagram outlining burst engine workflow.	24

LIST OF FIGURES (continued)

Figure	Page
2.10 Relation between spike arrival time-delta and synaptic weight: (a) In FEAGI, the difference between the burst sequence of the pre-synaptic and post-synaptic neuron determines the synaptic weight applied during long-term potentiation and long-term depression. Only if the difference of firing is one burst, will there be an impact, and for anything beyond the impact will be zero. (b) In depicting the typical STDP model, the difference between the arrival time of the pre-synaptic and post-synaptic spike (x-axis) determines the synaptic weight (y-axis).	27
2.11 Changes in neuron membrane potential across consecutive bursts: (a) A dip in the membrane potential has occurred since the neuron leakage has overpowered the influence of the upstream neurons. (b) In some cases, there is a perception of premature firing on the graph, which is due to the neuron fire logic occurring before recording the membrane potential in the connectome. (c) This area indicates of the span of a single burst.	28
3.1 Conceptual view of how memory is formed in FEAGI: process begins with raw data fed to IPU for initial processing and conversion to neuronal activity, then supplied to cortical pathways, and ultimately landing in memory region in form of cell assembly.	32
3.2 Conceptual view of how multiple memory formations can join to present same object.	33
3.3 Conceptual view of how memory recall can occur, assuming that artificial brain has been previously trained on two different forms of letter “a”, thereby forming two cell assemblies with a large number of neurons in common, as shown in Figure 3.2; exposure of a new form of letter “a” results in commonality of neurons between new cell assembly and the other two, thereby activating them and triggering memory recall.	34
3.4 Conceptual view of the flow of sensory information within FEAGI. An IPU is dedicated to each input type. IPU’s output is directed to functional pathways that differ in structure and number of layers from one type to another due to the differences like raw information under process. The end of each functional pathway leads to a memory region dedicated to it. Memory association occurs when events happen in close time proximity across memory regions regardless of type, enabling the framework to generate an OPU signal in one sensory type, even though the input stimuli was exposed to another.	35
3.5 Sequence diagram for memory formation.	37

LIST OF FIGURES (continued)

Figure	Page
4.1 Demonstration of how neuronal activities traverse through cortical layers during training and testing. Shaded blocks are an indicator of neuronal activity in the associated cortical layer (row) and burst instance (column). 39	
4.2 Concept of punishment as part of reinforcement learning implementation. In this example, number 6 is exposed to the system, but number 8 was detected. Due to the mismatch between the perceived number and the input number, pain receptors are stimulated to degrade the synaptic weights involved in producing the incorrect answer.	41
5.1 Structure of genome database.	43
6.1 FEAGI system-level workflow.	50
6.2 Monitoring dashboard created to help monitor cortical activities across various regions.	51
7.1 Visualization of cell assemblies. Each color cluster corresponds to the cell assembly associated with a particular number.	53
7.2 View of neuronal activities within the framework leading to association of UTF-8 version of character with handwritten version read from MNIST database.	55
7.3 Framework behavior during learning and recall: (a) exposure of handwritten version of number 1 along with UTF-8 version of number 1 to framework at same approximate time, leading to association of cell assemblies between visual memory and UTF-8 memory; (b) exposure of framework with another handwritten version of number 1 leads to activation of same cell assembly in UTF-8 memory region and recognition of character 1 in association with handwritten version.	57
7.4 Artificial brain's fitness improvement over generations. Individual dots outline the fitness value for a given generation; the line is a moving average over the past 100 data points; letters A, B, and C are indicators of a generation phase where a manually induced physiological mutation has occurred; letter D is where we introduced the early termination feature; letter E is where reinforcement learning was introduced; and the dotted line oval is highlighting the observation of less low-performing instances due to the implementation of the early termination feature. Data points with a fitness of zero have been excluded from the graph.	58

LIST OF FIGURES (continued)

Figure	Page
7.5 Relationship between total neuron count and disk space consumed by artificial brain.	62
7.6 Impact of exposure time on fitness and its correlations with the size of training set. For this analysis, a random genome was used frequently with a different number of training sets and exposure times. Each training set is considered 10 MNIST digits, and exposure time is the duration by which every training or test sample is exposed to the input processing unit.	63

LIST OF ABBREVIATIONS / NOMENCLATURE

AGI	Artificial General Intelligence
AI	Artificial Intelligence
CPPN	Compositional Pattern Producing Networks
CPU	Central Processing Unit
FEAGI	Framework (for) Evolutionary Artificial General Intelligence
HBP	Human Brain Project
IPU	Input Processing Unit
JSON	Java Script Object Notation
LGN	Lateral Geniculate Nucleus
LTD	Long Term Depression
LTP	Long Term Potentiation
NPU	Neural Processing Unit
OPU	Output Processing Unit
SDK	Software Development Kit
STDP	Spike Timing Dependent Plasticity

CHAPTER 1

INTRODUCTION

1.1 Background

Over the past decade, we have observed tremendous improvements in the field of artificial intelligence (AI). Google DeepMind’s AlphaGo [1] has been able to beat the world champion in the game of Go, and shortly after that a new version of the software called AlphaGo Zero was able to beat AlphaGo 100 to 0. In the medical field, deep learning is targeting many new applications. As an example, the use of **deep learning** for lung cancer screening [2] has outperformed radiologists. Without a doubt, AI is transforming our lives, but looking deeper into the type of problems solved, we observe that all advancements have been targeting a narrow set of applications; hence, they are referred to as “Narrow AI.” Artificial general intelligence (AGI), on the other hand, intends to be more flexible and generalizable and go beyond specific tasks. The ability to solve never before seen problems, thinking, making decisions, or planning for the future are just a few of the many capabilities that machines lack, in comparison to humans. AGI has long been envisioned to solve such problems and achieve human-level intelligence or beyond, but the progress in this field has been slow.

Many exciting brain research initiatives are in motion, both nationally and internationally, with a number of them focusing on methods and approaches to simulate the human brain as an approach to achieve artificial general intelligence. Despite decades worth of work, this field is still in its infancy, mainly due to the complexity of the human brain structure driven by the sheer number of neurons, variations in neuron morphology, and the dynamic nature of all components.

Billions of dollars are being spent on the simulation of the human brain, and it would be highly rewarding to see a new design paradigm accelerate the growth rate in this area.

Some research groups are approaching the challenge from a hardware perspective and developing neuromorphic architectures to provide a physical platform for brain simulation. Others are building software frameworks to leverage existing hardware and simulate brain behaviors using code. There are challenges to both approaches. On the hardware side, specially designed hardware needs to be paired with specialized algorithms to simulate a biological brain. Hardware by nature has rigidity in its architecture and its physical manifestation, hence making it almost impossible to evolve similarly to a biological brain. This limitation forces the evolutionary needs to be pushed to the software stack running on top of the hardware or even demanding a new hardware revision leading to enormous engineering and manufacturing expenses. On the other hand, software-based frameworks are very flexible and capable of evolving but are bound to the limitation of resources offered by the hardware on which they run. The biggest challenge on the software side is designing a network model that serves a desirable purpose.

In this research a new framework is proposed called the Framework for Evolutionary Artificial General Intelligence (FEAGI) to help us build intelligent systems inspired by the human brain, which can grow, evolve, scale, and adapt to the environment, requiring limited efforts to define structural characterizations for any given cortical area targeted for simulation. Our approach is a model based on the segregation of the biological brain's anatomy and physiology, which has enabled a simplified yet scalable design process.

As part of this dissertation, proposed framework's architecture and capabilities will be discussed in detail. The list of contributions are as follows:

- Design of a scalable framework architecture for human brain simulation based on spiking neurons.
- Novel method to indirectly encode an artificial brain's anatomical structures in a genome-inspired data structure through indirect encoding.

- Method to grow an artificial brain inspired by the process of neuroembryogenesis in the human brain.
- Modules with input and output interfaces to help convert physical stimuli into neuronal activities, and vice versa.
- Functions to simulate the biological brain’s physiological activities, such as neurotransmission, neuroplasticity, and inhibitory-excitatory behaviors.
- Special memory architecture to help exhibit cognitive abilities such as learning, long-term memorization, and memory recall.
- Genome database as part of the evolutionary framework to capture the artificial brain’s genomic information along with statistics tied to its performance to help us evolve the artificial brain’s anatomical properties over generations using genetic algorithms, thus allowing the artificial brain to improve its cognitive abilities over time.
- Burst management technique to alleviate the lack of parallelism present in an artificial environment that stems from software and hardware limitations compared to a biological counterpart.
- Augmented framework with a visualization module to help gain insight into the flow of information within cortical pathways as well as memory formations.

We believe that using such a framework can accelerate research in the quest for building large-scale models capable of artificial general intelligence. Our proof of concept demonstrates the ability to read digits from the MNIST database [3] and recognize them in real-time. To prove the versatility of the framework, after evolving the artificial brain through thousands of generations using handwritten digit training, we trained the new generations using apparel images from the Fashion-MNIST database [4].

1.2 State of the Art

Researchers have been pursuing human brain simulation with different objectives for many years, as shown in Figure 1.1. Some have quested to understand better how the human brain functions and to assist in root-causing brain-related disorders, and others have sought to create artificial intelligence (AI) modeled after the ultimate example provided by nature. Our goal in creating FEAGI is mainly the latter and to offer a framework that, on the one hand, can help grow a functional brain using predefined anatomical and physiological properties and, on the other hand, is capable of evolving to produce an artificial intelligence that is superior to that from which it was initially created.

Currently, there are multiple active brain simulation projects around the world. One of the most funded studies is the Human Brain Project (HBP) [5], with its goal of building a detailed working simulation of the entire human brain. The HBP is collaborating with a large number of scientists and collecting very detailed information about brain structures down to the synaptic level while leveraging supercomputers to run real-time simulations. The approach of the HBP is considered a bottom-up approach, producing results very close to the biological brain but associated with very high computing and modeling costs, complexity, and reliance on high-fidelity information from the biological brain [6].

The Semantic Pointer Architecture Unified Network, or Spaun [7], is a functional brain model with multiple cognitive abilities created using the neural simulator Nengo [8, 9] and built based on spiking neurons [10] by creating a higher-level model inspired by the biological brain. SPAUN is capable of performing eight tasks: copy drawing, image recognition, reinforcement learning, serial working, counting, question and answering, rapid variable creation, and fluid reasoning.

The Nengo framework offers a powerful software development kit (SDK) to design detailed artificial brain models. Nengo also can interface with neuromorphic hardware such as SpiNNaker [11] and Neurogrid [12].

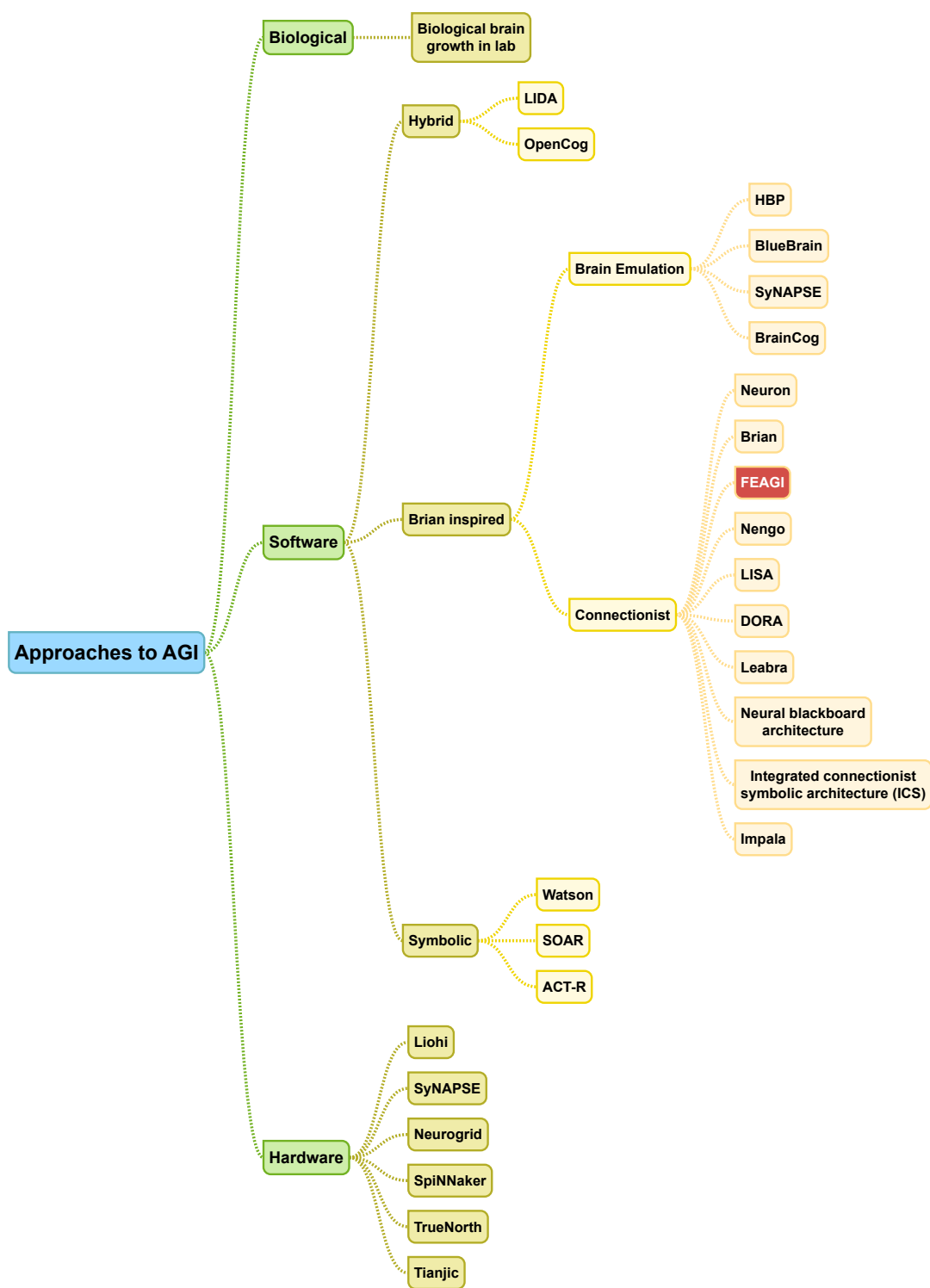


Figure 1.1. Diversity of approaches laying the foundation for AGI.

The Defense Advanced Research Projects Agency’s (DARPA) SyNAPSE project [13] has resulted in the creation of TrueNorth chip [14] accompanied by a software simulator called Compass [15, 16]. Compass is capable of simulating trillions of neurons on a super-computer. Other neuron-simulation software used by the research community includes NEST [17] and Brian [18], which are also designed for simulating spiking neuron models.

One of the key differences between existing brain simulators and the biological brain is the fact that a mature biological brain came into existence as a result of an enduring growing process referred to as “neuroembryogenesis.” In contrast, artificial deep neural networks simulating the brain are created using a predefined model. Our proposed framework is built on the novel idea of *growing* an artificial brain into existence using a genome-like structure that captures the biological brain’s anatomical properties, such as cortical geometries, neuron morphologies, and interneuron connectivities, through indirect encoding [19].

In 1992, Gruau [20] introduced cellular encoding to program neural networks genetically; ever since then, there has been a significant amount of research in this area. In 2003, Stanley and Miikkulainen [21] introduced the concept of artificial embryogeny as a subdiscipline of evolutionary computation, and later in 2007, Stanley introduced the compositional pattern-producing network (CPPN) [22], a particular kind of neural network acting as a function capable of generating realistic patterns. The CPPN, in conjunction with HyperNEAT [23], has proven to be very successful in evolving robot brains for improving motor behavior [23]. CPPN is created as a network of functions that can help produce phenotypes that follow patterns along with modularity and regularity. Phenotypes can be evolved using HyperNEAT [23, 24, 25] to produce new offspring.

Given all of the frameworks mentioned above, there is still a significant gap when considering each for designing an artificial general intelligence being. The approach adopted by HBP is low-level and outcome lacks system-level cognition. Nengo has cognitive abilities but requires written code that requires hours of simulation for seconds of real-time

behavior, thus making it challenging to evolve, scale and interact in real-time. HyperNEAT has the evolutionary recipe but lacks the architecture needed for implementing cognitive behaviors.

Here, FEAGI can offer a framework built from the ground and brain-inspired in every aspect. Our approach depends on moderate information about the biological brain's structural details to help us define the properties of each cortical region in the genome. Genome entries follow a standard grammar for all brain regions and have the potential for being interfaced with information collected by the HBP to build a more accurate initial seed for our evolutionary framework.

Given that our approach in designing the framework has been dominantly inspired by nature, a large amount of research was devoted to studying the human brain. We have studied the process of brain development starting with the formation of the neural tube all the way to postnatal growth, as shown in Figure 1.2 [26, 27]. As a result of our brain research, we have come to a few foundational realizations:

- The brain consists of many functional areas meshed together and work in synchrony.
- Each functional area is a complex multi-layer neural network.
- Neurons are not created the same, and they carry distinct morphologies due to the structure of their dendrites and axons.
- A mature human brain with cognitive abilities must go through many development phases to reach its final functional structure.
- Evolution has played a crucial role in shaping brain structures.
- Connectivity between neurons is intricate and vastly influential in how the brain processes stimuli.

- Plasticity plays a crucial role in memory formation. Here, new connections are formed, existing connections are strengthened or weakened, and some connections are eliminated.
- Without having a balance between excitatory and inhibitory neuronal interactions, the brain would not be able to constrain neuron stimulations, which would subsequently lead to excessive neuron firing.

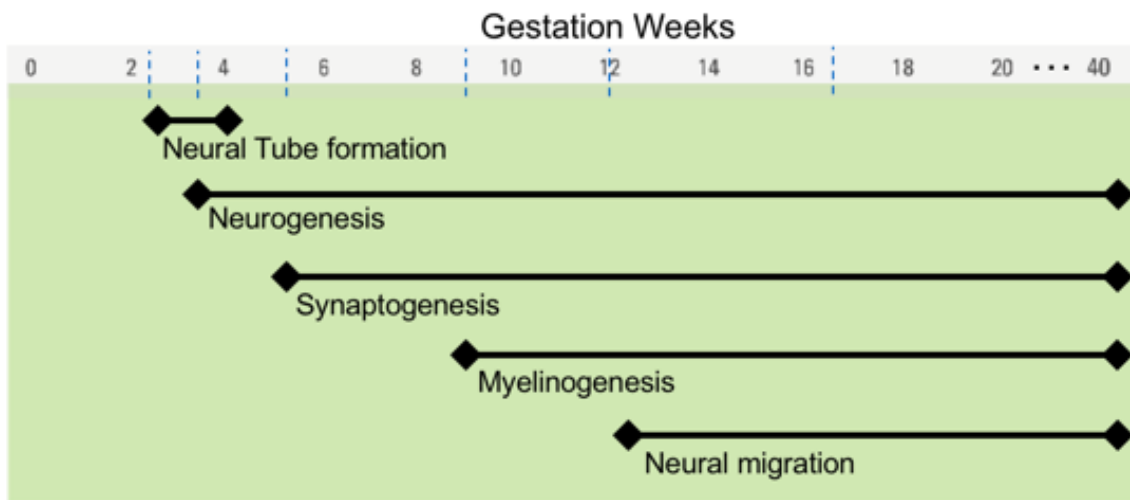


Figure 1.2. Timeline for key prenatal human brain development phases that are accounted for as part of framework design [27, 26, 28].

CHAPTER 2

FRAMEWORK ARCHITECTURE

2.1 Framework Overview

Our design methodology for the creation of the Framework for Evolutionary Artificial General Intelligence is based on an essential principle: to be inspired by the human brain but not to imitate it. The human brain is the ultimate machine regarding modularity, complexity, scalability, functionality, and evolvability; therefore, in order to build a framework capable of simulating its abilities, the qualities mentioned above must be taken into consideration, which is how we approached our design.

In summary, we have designed a framework capable of growing a plastic artificial brain that can manifest cognitive behaviors and evolve over generations. Here, we attempt to outline various components of the framework and how they relate to each other.

A genome-like data structure has been developed that uses an indirect encoding method to capture the human brain's anatomical properties as the genotype and helps to grow the artificial brain step by step by taking it through an artificial neuroembryogenesis process and storing it in another data structure, or "connectome" as a phenotype. The module responsible for this operation is called the "neuroembryogenesis unit."

After the anatomical structures are in place by completing the artificial equivalence of the neuroembryogenesis process, the artificial brain will be ready to receive stimuli from the outside world through a dedicated module called the "input processing unit" (IPU) and passing it through the corresponding cortical layers defined via the connectome. Interneuronal communications within the simulated brain uses an efficient burst centric mechanism inspired by the sparsity of neuronal activities in the biological brain to help mitigate the need for highly parallel processing units and time-dependent complexities associated with it so that the artificial brain can leverage commodity hardware while simulating a highly parallel structure to achieve the desired functionality.

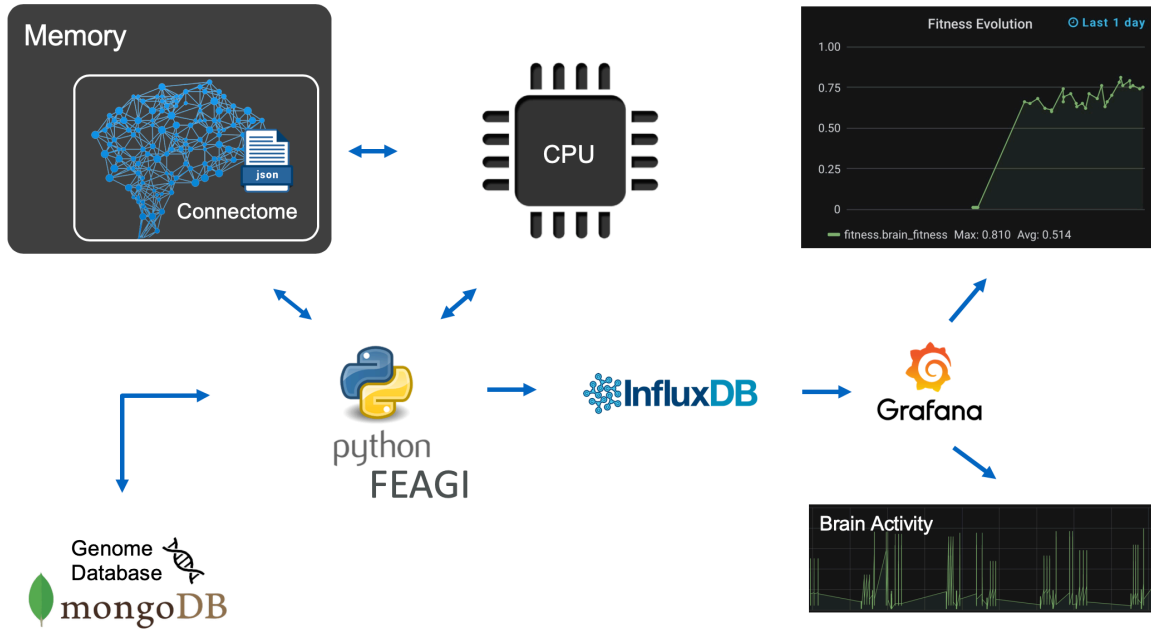


Figure 2.1. FEAGI’s software ecosystem.

Special functions have been designed as part of the “neural processing unit” (NPU) to simulate the biological brain’s physiological behaviors such as neuron firing, neuroplasticity, and neuron aging. The memory module has its unique structure to allow storage of the semantic pointers [29] generated from the information processed by cortical pathways. As its name implies, the learning module is responsible for learning activities, which will be described in detail in Section 3.5. This module is also capable of self-learning, which enables the framework to feed itself information in a particular manner to help learn on its own.

The evolutionary module uses genetic algorithms to evolve the working genome over generations based on the information it collects during the artificial brain’s lifecycle. Last, the activity visualizer module is developed to provide a better understanding of how data flows through the simulated brain and visualizes memory formations.

FEAGI itself is written in the Python programming language, but it is also integrated with other third-party software to leverage strength in storing data as well as data vi-

sualization, as shown in Figure 2.1. The MongoDB database is used to store genomes along with associated statistics, the InfluxDb database is used to record time-series data associated with the artificial brain’s interworking, and the Grafana monitoring tool has been used to visualize activities. The entire code base has been open sourced and can be downloaded at www.FEAGI.org.

2.2 Genome

In a biological organism, a genome is the collection of all genetic information needed to help the organism grow and function. Each individual piece of useful information as part of the genome is called a gene. Each gene has the capability of influencing a property or behavior for the organism. Structurally, a gene is a sequence of four distinct nitrogenous bases referred to as adenine (A), guanine (G), thymine (T), and cytosine (C). These four building blocks are called nucleotides. A double strand of these nucleotides paired as C–G or A–T base pairs become a DNA molecule.

In FEAGI, the term genome corresponds to a hierarchical data structure in the form of JavaScript Object Notation (JSON), a data-interchange file format. Subsequently, genes are defined in the form of key-value pairs as part of the JSON structure, as shown in Figure 2.2.

The genome is the key structure enabling us to evolve the artificial brain using genetic algorithms, as described in Section 5.2, and is used to capture information needed to be able to build and rebuild a functioning artificial brain. We have encoded anatomical properties –including cortical geometry, neuron density, neuron morphologies, growth rules, and orientation selectivity– as well as function variables driving the brain physiology –including firing threshold, plasticity constant, refractory period, leak coefficient, and maximum consecutive firing threshold– as genotypes using indirect encoding [23, 30]. The genome is read using the neuroembryogenesis unit and translated into cortical regions, cortical layers, neurons, and synapses between neurons as a phenotype that is

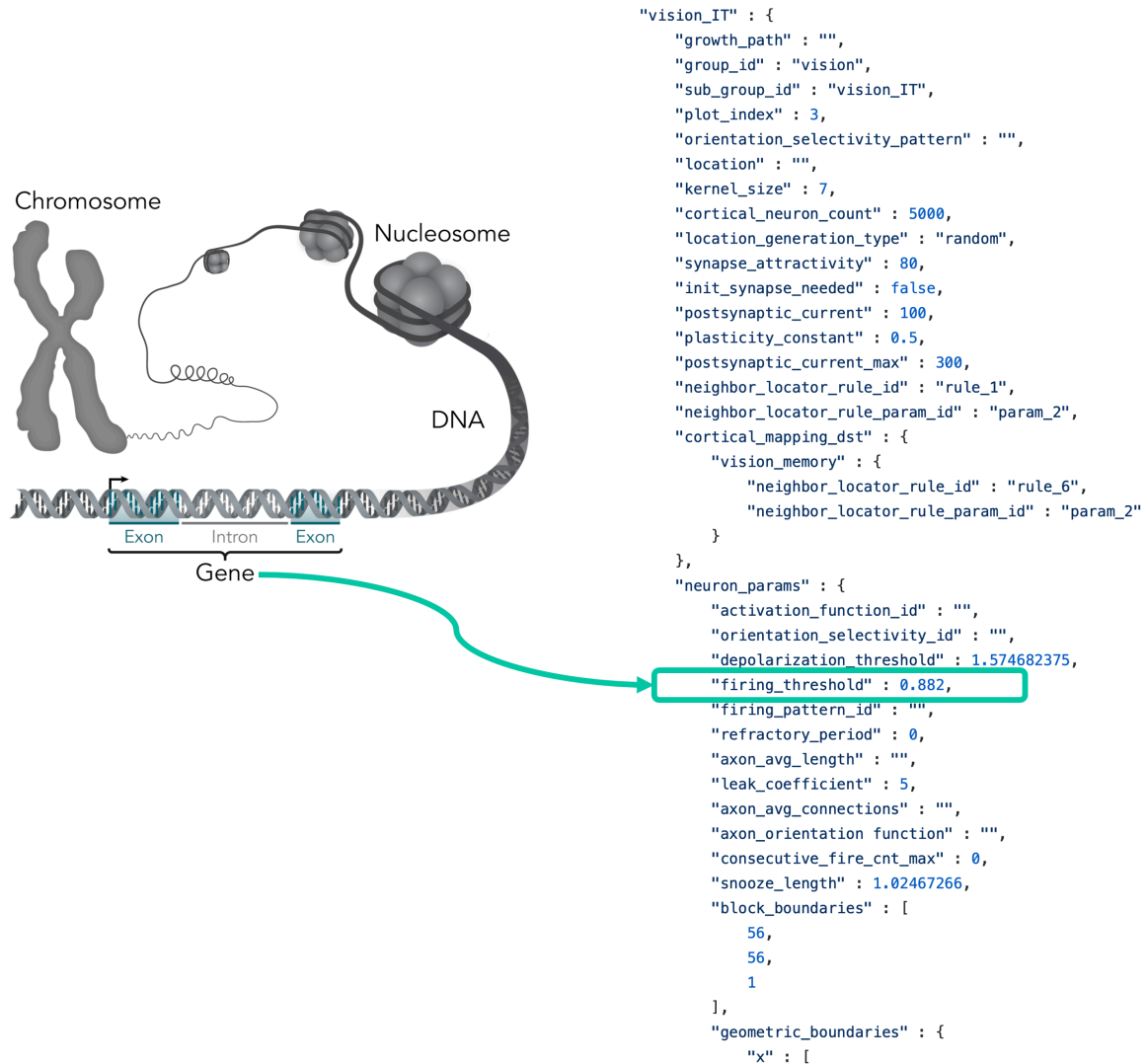


Figure 2.2. Relation between a biological gene and its counterpart in FEAGI.

stored in the connectome. Our genome implementation intentionally ignores anatomical structures that play a supportive role for the neurons such as the circulatory system, ventricles, meninges, and glial cells.

FEAGI generates the brain anatomical structures in its entirety using properties defined in the genome; therefore, without any code change and only through gene modification, a new artificial brain with a new set of properties can be generated. This code independence opens the door for using genetic algorithms to have them evolved over generations.

2.3 Connectome

The connectome is the phenotype created as a result of passing the genome through the artificial neuroembryogenesis process. The connectome is built using an object-oriented data structure to support scalability requirements when the implementation reaches the full human brain scale. The connectome contains information on individual cortical areas, cortical layers, neurons, neuron locations, synapses between neurons, and connection strengths.

2.4 Neuroembryogenesis Unit

Development of the biological brain is a very complicated and enduring process starting from the early stages of embryonic development called “neurulation,” where the neural tube takes shape, and continues to evolve during postnatal development. In the upcoming subsections, we describe how our framework is structured to simulate the processes involved in human brain development. In FEAGI, we have been inspired by the biological process and implemented a module that orchestrates a simplified version by reading instructions from the genome and recording the phenotypes in our connectome, as shown in Figure 2.3. By the end of this process, we will have cortical areas, layers, sublayers, neurons, and synapses in place.

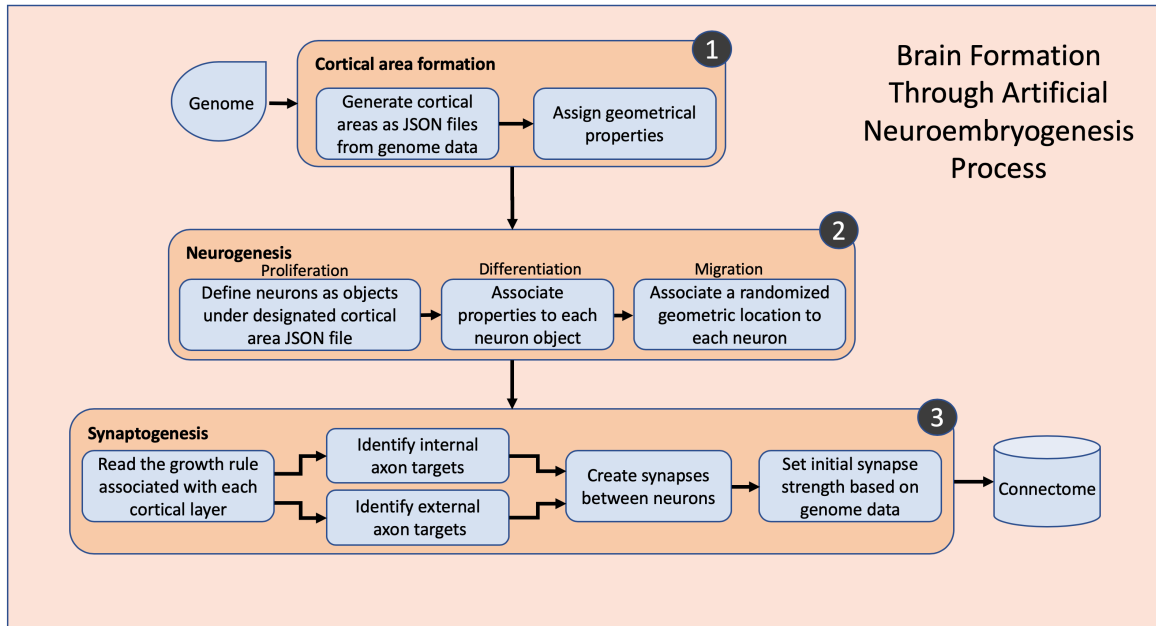


Figure 2.3. Artificial brain formation that begins by reading genome instructions and going through multiple phases sequentially, eventually capturing neurons and synapses that are created throughout the process in an object-oriented database, or connectome, for future use as part of the artificial brain’s operations.

2.4.1 Cortical Area Formation

The neuroembryogenesis unit is developed as a collection of procedures responsible for decoding the genome and building up the connectome. The first step is to have the functional cortical areas defined. In our implementation, every functional cortical area consists of multiple regions and subregions. As an example, to implement the ventral visual pathway, we have defined areas including V1, V2, V4, and IT, as cortical regions, and for region V1, we have defined subregions to accommodate the presence of various cortical layers. This structure has helped us to account for differences in characteristics across distinct cortical layers.

2.4.2 Neurogenesis

The next step after defining the cortical areas is to populate them with neurons. Our neurogenesis implementation covers biological phases including neuron **proliferation**,

migration, and **differentiation** [31, 32]. Since we are not dealing with a physical environment and all definitions are logical, we have adopted a simplified approach to the phases mentioned above: First, the genome is read to decode the density of neurons in a given cortical area. Next, we add neurons in the form of a JSON object to the connectome data structure –referred to as “proliferation” in the biological brain– while associating neuron properties in the form of key-value pairs to each neuron object inherited from the corresponding cortical layer –referred to as “differentiation” in the biological brain. For the final step, we assign each neuron a fixed location in the form of $[x, y, z]$ relative to the boundaries of the cortical region’s geometry –referred to as “migration”[33].

Due to the diversity of neuron types in the human central nervous system, as part of the design of the framework, we anticipated that support for different neuron models would be essential. In our implementation, we developed a custom neuron model that at large follows the leaky integrate and fire model [34]. The framework is designed to allow each cortical layer or sublayer to have its own designated neuron model. This implementation enables us in the future to model various cortical areas of the biological brain that do not follow a standard neuron model. Parameters defining the neuron model and its behavior are encoded in the genome.

Here are the highlights of how neurons are defined in FEAGI:

- Every neuron is created as a result of the neurogenesis process and is identified as an object in the connectome database.
- Metadata are associated with each neuron in the database, capturing neuron as well as synaptic properties.
- In the biological brain, two neurons can share hundreds or thousands of synapses. In FEAGI, we have defined a single aggregate that represents the level of impact of one neuron on another. This design decision simplifies the overall architecture,

saves storage, and improves performance to a large extent by eliminating the need to track and process a large number of connections between two individual neurons.

- Synapse definition is established by the association of the destination neuron identification number with the source neuron object in the database along with a value defining the synaptic strength among them.
- When the synapse between a source and destination neuron is initially created, a default post-synaptic current value is associated with the source neuron's axon. This value will later change as the neuron goes through long-term potentiation (LTP) or long-term depression (LTD), which is described in detail in Section 2.6.3.
- Upon neuron firing, its post-synaptic current will create a change in the membrane potential of the downstream neurons. If the neuron is defined as excitatory, then will cause an increase, and if the neuron is defined as inhibitory, then it will cause a decrease.
- The leaky behavior of the neuron is implemented such that with the passage of each burst, a neuron loses some of its membrane potential.

2.4.3 Synaptogenesis

Neighbor identification is the prerequisite to synaptogenesis. Axonal growth in a biological brain is an intricate yet exquisite process. An axon's growth is guided step by step through molecular cues, with some being attractive and others repulsive [32]. Sometimes they travel very long distances and surprisingly find their target destination with precision [32, 35]. Axon projections from one cortical layer to another or from one region to another are responsible for shaping the connections within the network and are essential in how data is passed along the cortical pathways and how information is processed.

In our framework, since we are dealing with a digital ecosystem instead of a biological one, we have explored alternative solutions. We investigated various neuron morpholo-

gies and noticed that the synapse locations on an axon could be enclosed and defined by a three-dimensional geometrical space outlining a region where the axon of one neuron would synapse with other neurons with high probability.

We have defined a set of geometrical constraints and called them "growth rules." These rules consist of a set of formulas defining spatial boundaries where synaptogenesis is allowed to occur, as shown in Figure 2.4. To find candidate neurons, we first read the location of the source neuron, target cortical region, and growth rule applied to the neuron from the genome. The growth rule helps us identify all candidate neurons from the target cortical region by checking if the target neuron coordinate falls within the region defined by the growth rule. The final step is to select a subset of neurons from the pool of eligible neurons to create the synapses with. For this step, we randomly select n neurons from the candidate pool, where n is a parameter defined in the genome for the source neuron's cortical area. With this approach, we have enabled the framework to guide axons to their destination, similar to how biological neurons find their own. Parameters associated with these rules are added to the artificial brain's genome so they can evolve. After the neighbor identification process detects candidate neurons for a given source, we can generate a synapse between the two, by adding the target neuron's identification number and synaptic weight to the source neuron's object in connectome.

2.5 Input/Output Processing Unit

Input and output processing units are the interfaces between the artificial brain and the outside world. Input processing modules are designed to receive raw data from various input portals and have that information ready to feed to cortical pathways in the form of neuron stimulations. Similarly, to be able to understand the artificial brain's output, either by humans or a computer, neuronal activities must be processed and relayed to the corresponding output device.

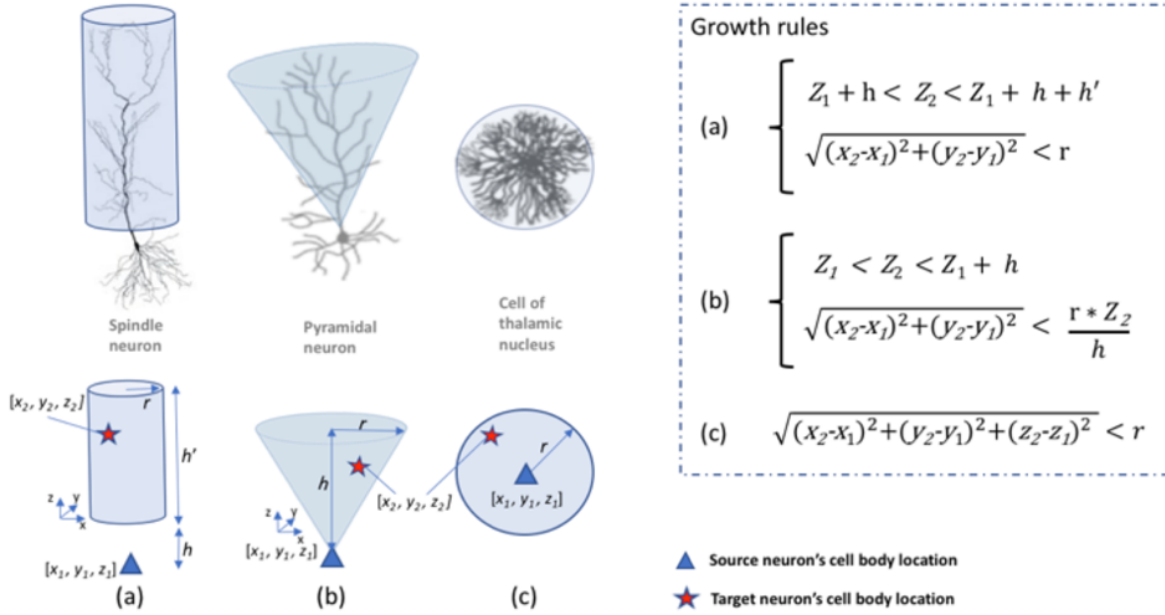


Figure 2.4. Growth rules as a set of geometrical formulas defining space in which axon of source neuron can find its destination neuron, defined in Cartesian coordinates and inspired by morphology of biological neurons; source neuron's coordinates represented by $[X_1, Y_1, Z_1]$ and a potential destination neuron's coordinate represented by $[X_2, Y_2, Z_2]$: (a) growth rule "a" defines a cylindrical boundary into which the destination neuron coordinates can fall, where h is neuron's distance from the cylinder's base, h' is height and r is radius of cylinder; (b) growth rule "b" simulates cone-shaped synaptogenesis region similar to that occupied by pyramidal neuron's axons, where h is height of the cone, and r is radius of its base; (c) growth rule "c" represents spherical-shaped neuron with axon length reaching maximum r as sphere's radius.

The biological brain receives information from the peripheral nervous system connected to various receptors such as photoreceptors, thermoreceptors, chemoreceptors, and mechanoreceptors. Similarly, in FEAGI, every input processing unit/output processing unit (IPU/OPU) would differ in how information is collected, encoded, and presented to the first cortical layer as part of a particular functional cortical pathway. As an example, for visual information, there will be a designated IPU that will convert the image of a given object to its fundamental elements such as color and brightness while maintaining the pixel arrangements, similar to the retinal information-processing structure. The human eye collects optical information using photoreceptors and passes it through an as-

sortment of cells before forwarding it to the thalamus via ganglion cells [32]. Additional mapping occurs through the six layers of the lateral geniculate nucleus (LGN) within the thalamus, as shown in Figure 2.5. In the proposed framework, the visual IPU plays the same role as the retina, and the rest of the processing activities are handed off in the form of neuronal stimulation to the neural processing unit representing the cortical pathways.

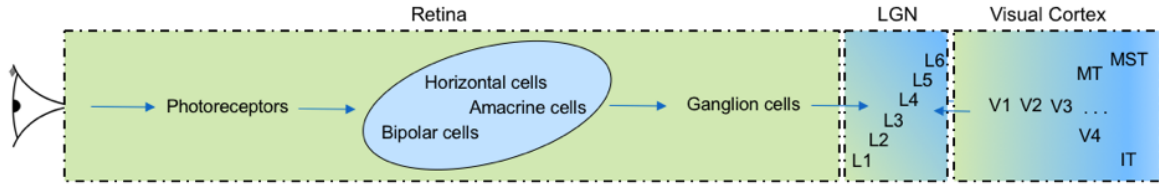


Figure 2.5. Visual pathway in human brain [32].

As part of the design of the vision IPU, we have applied two fundamental principles inspired by nature. The first is based on how neuron projections maintain their position fidelity against their neighbors as they project from the retina to LGN, and from there to the striate cortex. The second principle corresponds to how horizontal cells with dendrites spanning sideways collect information from neighboring neurons. We have applied this principle by leveraging a well-known technique in image processing that involves applying a kernel convolution matrix by which a small-size matrix is used to scan the image while applying filters. This method helps with breaking down the image into its fundamental components such as orientation, color, brightness, and contrast. Sensitivity to these fundamental factors leads to the activation of neurons at the beginning of the visual cortical pathway, as shown in Figure 2.6.

The process of decoding memory back to output processing unit first requires proper neuron wiring from the memory region to the OPU, which would be driven by the genome. The second step is the actual decoding of neuron signals received from its corresponding memory unit to the actual physical signal needed to interact with systems that are external to the artificial brain. In our proof of concept, we have only implemented the UTF-8 OPU.

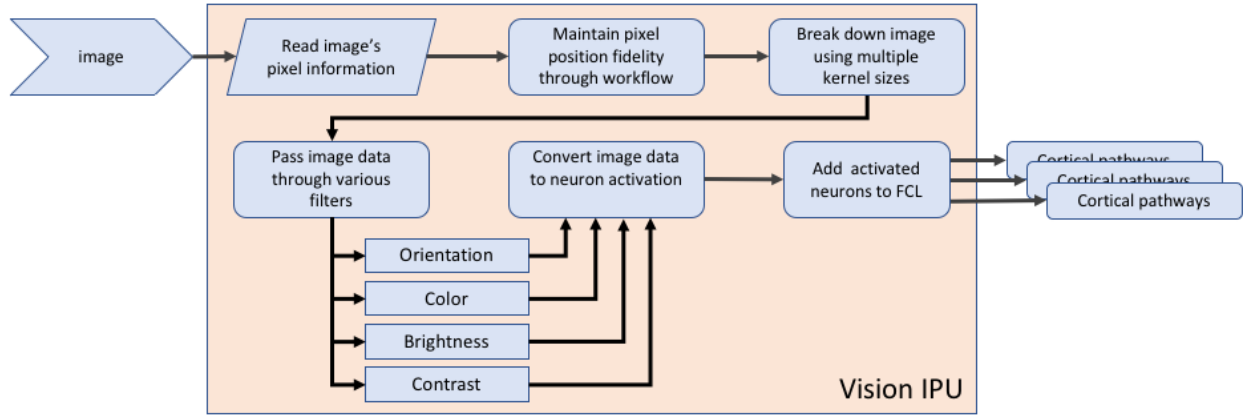


Figure 2.6. Flow of information through vision IPU.

The neuron wiring for UTF-8 IPU was a one-to-one neuron mapping from the memory region to a one-dimensional matrix representing individual UTF-8 characters, as shown in Figure 2.7.

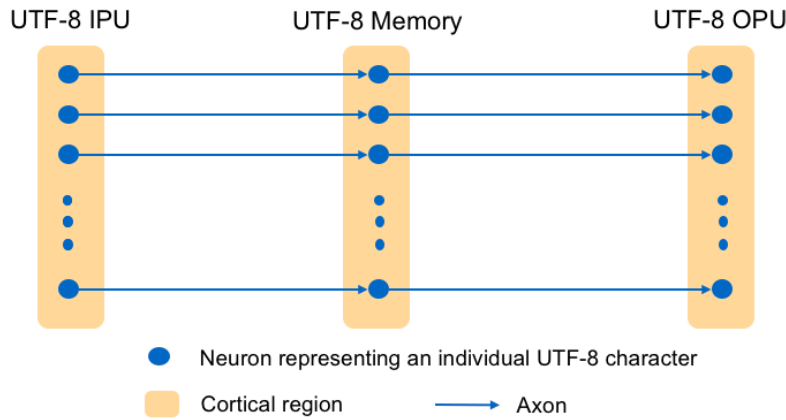


Figure 2.7. Connectivity between cortical layers representing UTF-8 module.

2.5.1 Transforming an Image Using Kernel Convolution

Pixels are the smallest units of data in a digital image, each of them containing properties such as the position in reference to an origin and color. An image can also be broken down into segments larger than a single pixel by defining a block size consisting of multi-

ple pixels. Defining blocks would allow a higher level analysis of picture properties, such as edges and contrast delineation between neighboring points.

The kernel convolution method is based on running a small matrix, referred to as a kernel, through every single pixel of the image and generating a new image as a result. Depending on the construct of the kernel matrix, the output image can be drastically different. Kernels can be purposefully built to perform a particular transformation on the image, such as exposing the edges, fading, or applying orientation selectivity to extract edges with a specific direction. For example, the following kernel matrix can be applied to an image to extract edges with a 45° angle.

$$\begin{bmatrix} -1 & -1 & 1 \\ -1 & 1 & -1 \\ 1 & -1 & -1 \end{bmatrix}$$

Kernel matrices can vary in size, which dictates the scope of its influence when applied to a particular pixel of the image. For example, a 3×3 kernel matrix influences a single layer of pixels around a target. Respectively, a 5×5 kernel would impact two surrounding layers. As a general rule, an $n \times n$ kernel impacts $\frac{(n-1)}{2}$ layers around a target pixel, which is why the kernel dimensions are always chosen with odd numbers.

After performing kernel convolution against an image, it is common to feed the transformed image through a filter operation to remove elements below a specific threshold. This action, also known as the activation threshold for neurons, will help remove noise from the transformed image.

The final step before feeding image data to the spiking neural network is to multiply the results by an amplification constant to help adjust the magnitudes to the level suitable for activating the spiking neurons.

2.6 Neural Processing Unit

The NPU is where the majority of the artificial brain’s physiological behaviors are managed and orchestrated, while anatomical definitions are all driven by the connectome. The NPU consists of a collection of functions that are described as follows:

2.6.1 Neuron Firing, Queues, and Bursts

There are approximately 100 billion neurons in a mature human brain, and the timely firing of individual neurons drives the transferring and processing of information across the brain. There are two significant challenges to simulating the brain; one associated with the brain’s architecture stemmed by how neurons are interconnected, and the other tied to the parallel processing nature of the brain since every neuron can act independently of others. To address the latter, we have developed a method called “burst engine.”

The concept of burst engine is based on event-driven simulation models [36, 37, 38, 39] and the underlying principle of how FEAGI processes information. Every burst instance is associated with a list of neurons, in the form of a queue, whose membrane potentials have passed their firing thresholds and are ready to fire. This list is referred to as the “fire candidate list” or FCL. The burst engine fires all neurons listed in the FCL and simultaneously updates the membrane potential of the neurons, which have synaptic connectivity with those that are firing, as shown in Figure 2.8 and outlined by Algorithm 2.1. Subsequently, the burst engine workflow is captured by Figure 2.9.

Synaptic information is queried from the connectome through dictionary lookups. If the membrane potential of any of the destination neurons exceeds their threshold, they will be added to the FCL and fired in the next round of the burst engine cycle. At any point in time, we maintain two instances of the FCL, one of which is the previous instance. Comparing the content of these two lists has enabled us to implement neuroplasticity, which is explained in detail in Section 2.6.3. Managing neuron firing through bursts has helped us gain control over the highly parallel nature of spiking neurons, which play a

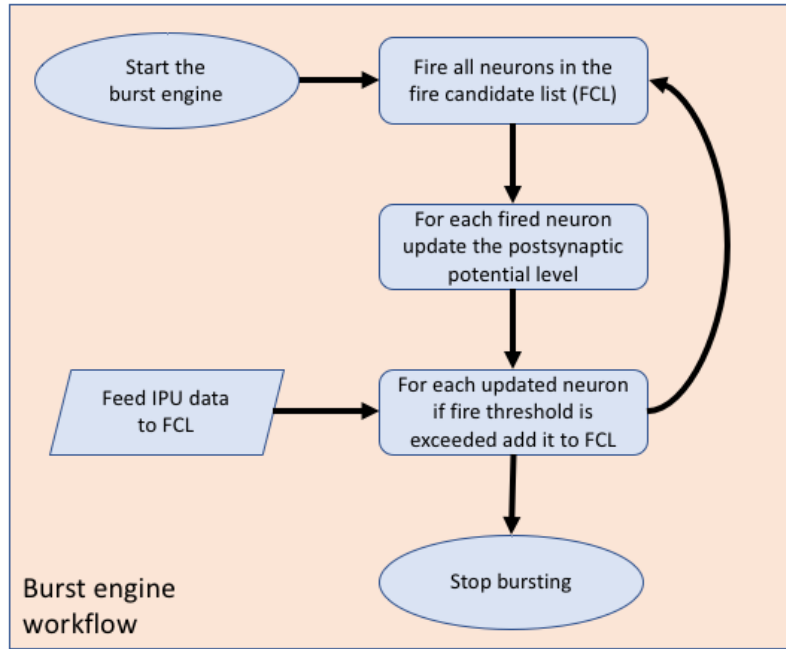


Figure 2.8. Artificial brain built by FEAGI running on bursts, whereby list of neurons ready to fire is added to fire queue, and they all fire simultaneously when burst is triggered, a mechanism used to overcome the parallel nature of spiking neurons when implemented in environment with limitations in parallelism.

Algorithm 2.1 Burst Engine

```

1: procedure BURSTENGINE
2:   while burst_exit_criteria=False do
3:     if training_mode = True then
4:       inject_training_data()
5:     if test_mode=True then
6:       inject_test_data()
7:     for neuron in fire_candidate_list do
8:       fire_neuron(neuron)
9:     neuroplasticity()
10:    cortical_activity_during_training = measure_cortical_activity()
11:    if cortical_activity_during_training < activity_threshold then
12:      burst_exit_criteria ← True
13:    save_brain_instance_statistics_in_database()

```

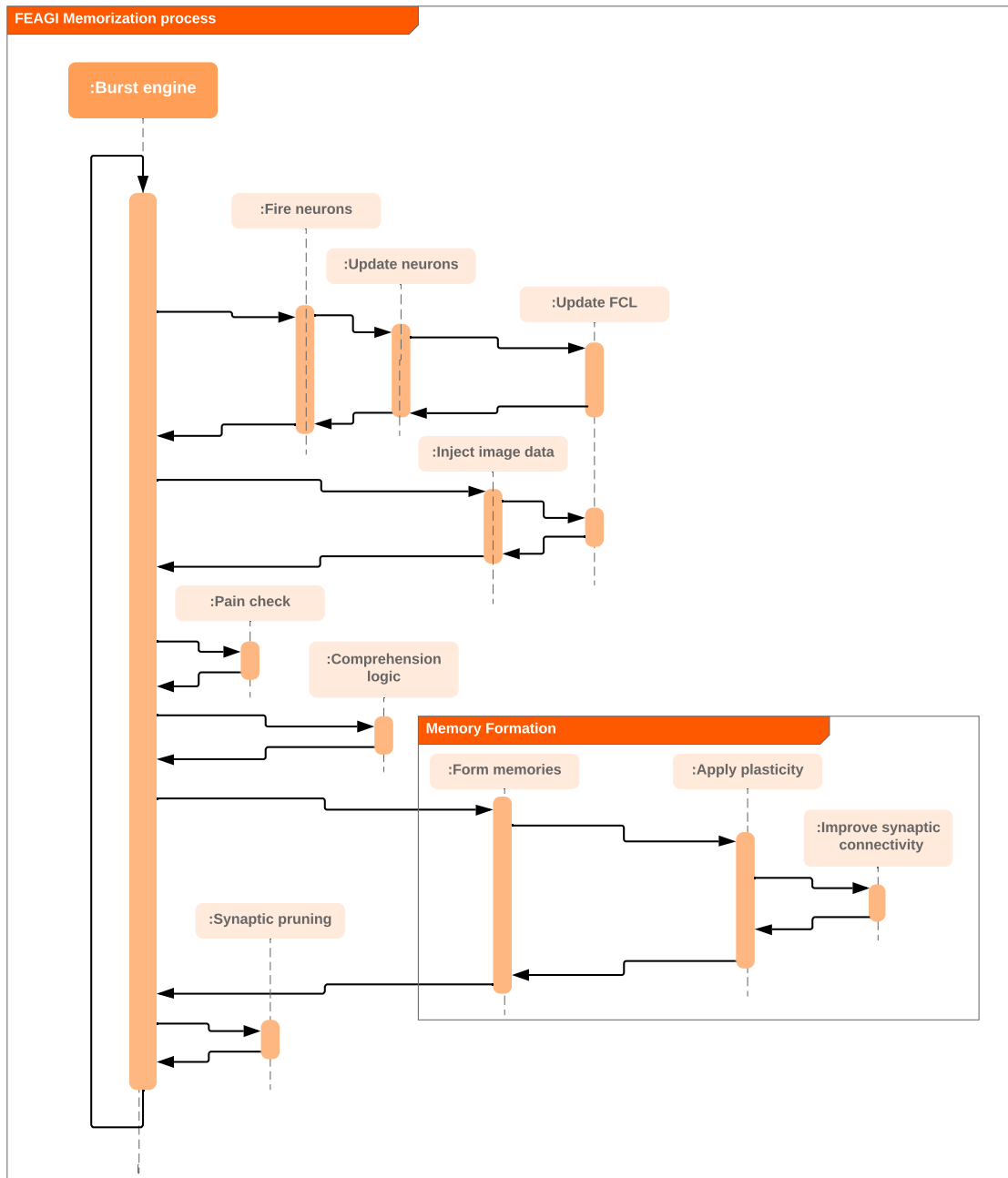


Figure 2.9. Sequence diagram outlining burst engine workflow.

crucial role in learning and memorization. We have analyzed the algorithmic complexity of our burst engine logic and have evaluated it to be $O(n^2)$. This complexity is associated with the number of firing neurons times the average number of synapses associated with each of them. It is important to note that the total number of neurons is not a contributing factor.

2.6.2 Neurotransmitters

In the biological brain, after the neuron firing occurs, depending on the type of neurotransmitters released by the pre-synaptic neuron, different types of behavior are observed on the post-synaptic neuron. FEAGI accounts for both excitatory and inhibitory neurotransmitter effects. We have defined excitatory behavior to update the post-synaptic neuron properties within the connectome to show an increase in action potential every time the upstream neuron fires.

In our implementation, the inhibitory behavior takes two different forms: the first is based on the inhibitory synapse type, where upon firing, the firing neuron would have a negative impact on the action potential of the downstream neuron; the second form keeps track of the number of consecutive firings of a neuron within a window of bursts, and upon exceeding the threshold, a snooze flag is activated, thus preventing neurons from firing for a predefined number of bursts as defined in the genome. Implementation of this inhibitory behavior has proven to be very beneficial since we observed that without it, neuron clusters were entering an infinite loop of firing. The inhibitory function acts as a damper, thereby regulating excessive feedback and stabilizing the communication.

2.6.3 Neuroplasticity

Hebb's hypothesis [32] famously states that "Neurons that fire together, wire together." We have implemented this hypothesis as a variant implementation of spike-timing-dependent plasticity (STDP). In traditional STDP models [40], there is a continuous relationship between synaptic weight and time-delta between the arrival of the pre-synaptic spike and

the post-synaptic spike, that is, the longer the time-delta, the lower the synaptic weight of the STDP. In our implementation, we have leveraged the sequence of bursts produced by the burst engine and kept the window of influence down to a single burst sequence, as shown in Figure 2.10, and the mathematical formula deriving the first graph is represented by (2.1).

$$SynapticWeight = \begin{cases} C & \text{if } \Delta B = +1 \\ -C & \text{if } \Delta B = -1 , \\ 0 & \text{if } |\Delta B| > 1 \end{cases} \quad (2.1)$$

where C is the potentiation constant defined as a gene for each cortical layer, ΔB is the difference between the burst sequence identifier associated with the post-synaptic firing neuron B_{post} and the pre-synaptic firing neuron B_{pre} , as shown in equation (2.2).

$$\Delta B = B_{post} - B_{pre} \quad (2.2)$$

Assuming the upstream neuron A has its axon connected to neuron B , in STDP, when A fires briefly before B , this leads to LTP as outlined by Algorithm 2.2, and when A fires briefly after B , this leads to LTD on the synapse between A and B , as outlined by Algorithm 2.3 . In our implementation, when A fires in burst n and B fires in burst $n + 1$, this leads to LTP, and when A fires in burst n and B fires in burst $n - 1$, then LTD occurs. In other terms, when a post-synaptic neuron fires in burst $n - 1$ and a pre-synaptic neuron fires in burst n , there will be a penalty on the strength of the connection between the two neurons.

Algorithm 2.2 Long-Term Potentiation

- 1: **procedure** APPLYLONGTERMPOTENTIATION(*source_neuron*, *destination_neuron*)
 - 2: *synaptic_strenght*(*srouce_neuron*, *destination_neuron*) += *plasticity_constant*
 - 3: **if** *synaptic_strenght* > *synaptic_strenght_max* **then**
 - 4: *synaptic_strenght* $\leftarrow$ *synaptic_strenght_max*
-

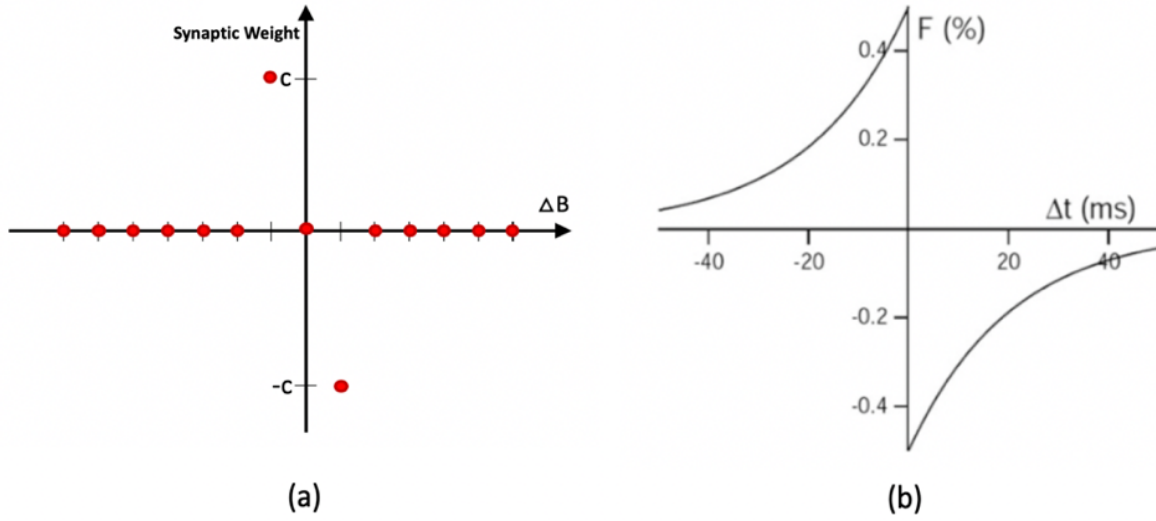


Figure 2.10. Relation between spike arrival time-delta and synaptic weight: (a) In FEAGI, the difference between the burst sequence of the pre-synaptic and post-synaptic neuron determines the synaptic weight applied during long-term potentiation and long-term depression. Only if the difference of firing is one burst, will there be an impact, and for anything beyond the impact will be zero. (b) In depicting the typical STDP model, the difference between the arrival time of the pre-synaptic and post-synaptic spike (x-axis) determines the synaptic weight (y-axis) [40].

Algorithm 2.3 Long-Term Depression

```

1: procedure APPLYLONGTERMDPRESSION(source_neuron, destination_neuron)
2:   synaptic_strenght(srouce_neuron, destination_neuron) -= plasticity_constant
3:   if synaptic_strenght < synaptic_strenght_min then
4:     synaptic_strenght  $\leftarrow$  synaptic_strenght_min

```

2.6.4 Spiking Neuron Model

The neuron model is a mathematical and logical description defining a neuron's behavior. Many neuron models have been proposed and studied over decades [41]. In our implementation, we have adopted a custom model that is largely designed based on the well-known leaky integrate and fire model [34]. Given we have adopted a burst centric logic for our framework, this design decision has fundamentally influenced the underlying neuron model, as shown in (2.3):

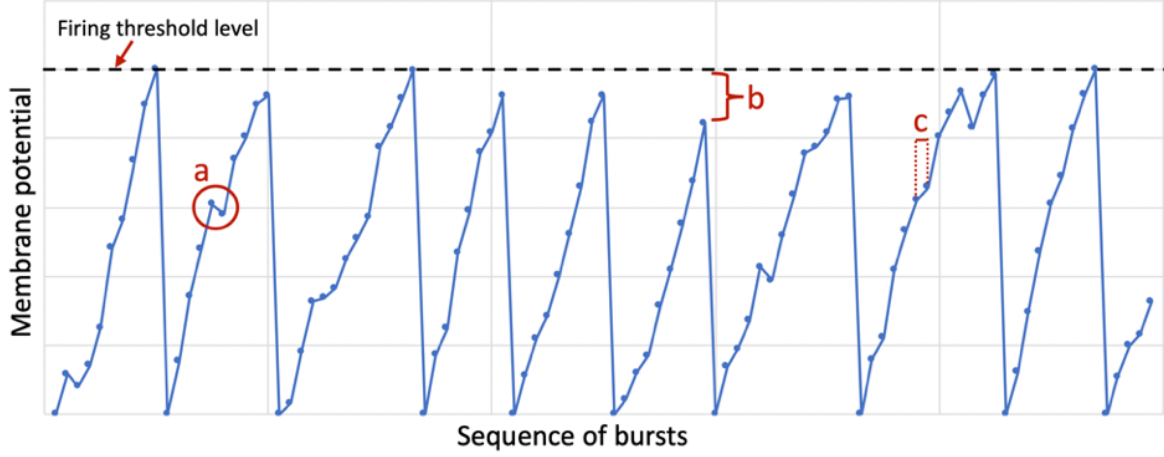


Figure 2.11. Changes in neuron membrane potential across consecutive bursts: (a) A dip in the membrane potential has occurred since the neuron leakage has overpowered the influence of the upstream neurons. (b) In some cases, there is a perception of premature firing on the graph, which is due to the neuron fire logic occurring before recording the membrane potential in the connectome. (c) This area indicates of the span of a single burst.

$$P = \sum_{i=1}^n I_i - \frac{L_b}{L_c}, \quad (2.3)$$

where P is the membrane potential for the neuron, I_i is the post-synaptic current of the i^{th} upstream neighbor, L_b is the number of bursts since the last neuron membrane potential calculation, and L_c is the leak constant. Equation (2.4) describes the neuron fire trigger condition:

$$(P > F_t) \wedge (F_b > R_b) \wedge (C > S), \quad (2.4)$$

where P is the membrane potential, F_t is the firing threshold, F_b is the number of bursts since the last neuron firing, R_b is the neuron's refractory period in the form of burst count, C is the pointer identifying the current burst counter position, and S is the pointer identifying the burst counter position by which the neuron was flagged in order to be snoozed. Neuron firing is outlined by Algorithm 2.4, and the dynamics in membrane potential across bursts are depicted in Figure 2.11.

Algorithm 2.4 Neuron Firing

```
1: procedure NEURONFIRING
2:   for synapse in firing_neuron_synapse_list do
3:     update_destination_membrane_potentials()
4:     if destination_membrane_potential > firing_threshold then
5:       add_destination_neuron_to_fir_candidate_list()
6:   update_counters_associated_with_firing_neuron()
7:   if consecutive_fire_counter > consecutive_fire_counter_threshold then
8:     start_refractory_period_for_firing_neuron()
9:   remove_firing_neuron_from_fire_candidate_list()
```

Considering the leaky nature of the neuron membrane potential, when this value is being updated due to upstream activity, the associated leak value must be calculated so it can account for part of the final value of the neuron membrane potential. This process is outlined by Algorithm 2.5.

Algorithm 2.5 Updating Membrane Potentials

```
1: procedure MEMBRANEPOTENTIALUPDATES
2:   leak_value  $\leftarrow$  calculate_neuron_potential_leakage_since_last_update()
3:   membrane_potential  $\leftarrow$  old_potential - leak_value + incoming_potential
```

2.6.5 Refractory Period

The refractory period in the biological brain is a duration post-neuron firing, whereby the neuron would be unable to fire again until the period is over. In our implementation, instead of the absolute time, we have used the concept of bursts in a way that after a neuron fires, we prevent it from firing for a predefined number of bursts, as outlined by Algorithm 2.6. The refractory burst count is defined for a given neuron as part of the neuron properties in the genome for a given cortical layer/sublayer, and its value can evolve over generations.

Algorithm 2.6 Refractory Period

```
1: procedure REFRACTORYPERIOD
2:   if current_burst_sequence_counter <
3:   (refractory_period+last_neuron_fire_burst_sequence) then
4:     Return True
```

2.6.6 Synaptic Pruning

As the brain operates, neuroplasticity causes synaptic weight among neurons to increase or decrease. In some cases, the synaptic weight between two neurons may reduce to a very small value or zero. Such synapses will be eliminated. This is synonymous to the synaptic pruning phenomenon in the biological brain.

CHAPTER 3

MEMORY AND LEARNING

Both memory and learning are closely interrelated and together play a crucial role in cognition and intelligence. In this research, we have mainly focused on long-term declarative memory and have deferred coverage on other memory types to future research. Many structures within the human brain are involved in declarative memory formation, and depending on the input type, different cortical areas play a role.

3.1 Memory Design

Donald Hebb, in a book published in 1949 [42], proposed the concept of “cell assembly” as a group of simultaneously activated neurons representing an object in response to an external stimulus. We have leveraged this concept as the core foundation of our memory design.

3.2 Memorization Process

When an input processing unit is exposed to an external stimulus, it triggers a set of chain reactions. First, raw information is broken down by the IPU using functions that are sensitive to the fundamental physical properties of the stimuli, as explained in Section 2.5. For example, a sound will be broken down into frequency and magnitude modulation bands, and an image will be broken down into contrast and color variation before they are fed back into their designated functional pathways, as shown in Figure 2.6. Functional pathways are a collection of interrelated cortical layers with different neuronal properties working together to process the information further. At the end of the pathway, a semantic pointer [29] will represent the external stimulus as a unique signature and then have it projected to the memory region. As the memory neurons ensemble becomes

activated in close time proximity, long-term potentiation (LTP) occurs among them, thus leading to a strengthened synaptic connectivity among neurons tied to the stimulus. This ensemble is what we refer to as a cell assembly, as shown in Figure 3.1. Given that the activation of memory neurons by upstream cortical layers is an asynchronous process, the formation of the cell assembly is gradual and along the way, leads to the increase in synaptic connectivity between neurons, but not all to the same extent. In general, neurons participating in a cell assembly are not fully connected, but in rare cases, this could be a possibility.

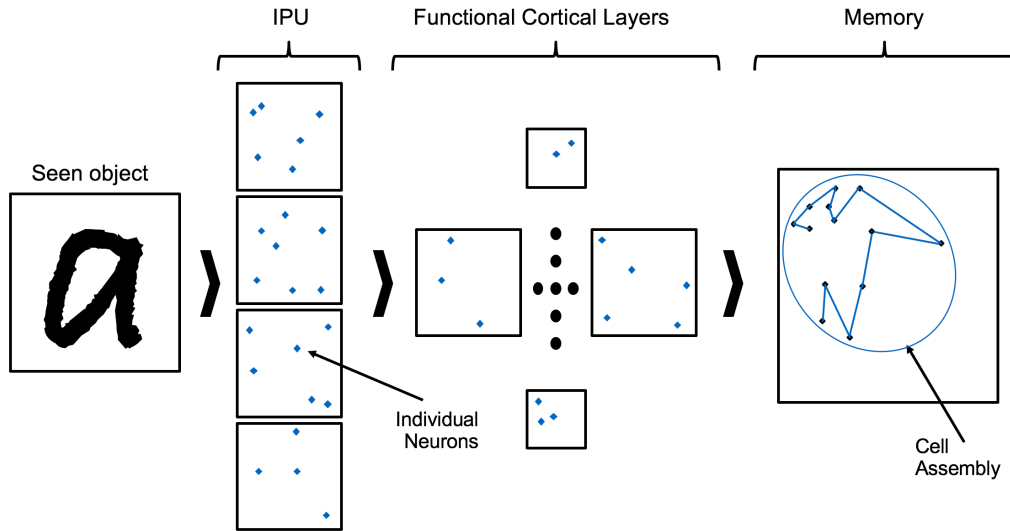


Figure 3.1. Conceptual view of how memory is formed in FEAGI: process begins with raw data fed to IPU for initial processing and conversion to neuronal activity, then supplied to cortical pathways, and ultimately landing in memory region in form of cell assembly.

3.3 Declarative Memory

Declarative or explicit memory is a type of memory related to events or facts and can be consciously recalled [32]. Declarative memory can be classified into working short-term memory and working long-term memory. Long-term memory is the one we implemented as part of the framework. In our implementation, long-term memory consists of

a pool of loosely connected neurons. When an object stimulates the IPU, some neurons in the memory pool become activated; when another similar object is exposed, another set of neurons becomes activated. When both objects are introduced to the IPU in a short period, a large percentage of activated neurons will be common between the two events, due to event similarities. This situation is where the enhancement of synaptic strength between neurons belonging to the cell assemblies associated with each object occurs. After multiple occurrences of seeing both objects simultaneously, the strengthened synaptic connection between neurons would tie the objects together. Therefore, if the stimulus related to the first object occurs, then it would lead to activation of neurons related to the co-occurring object as well, as shown in Figure 3.2.

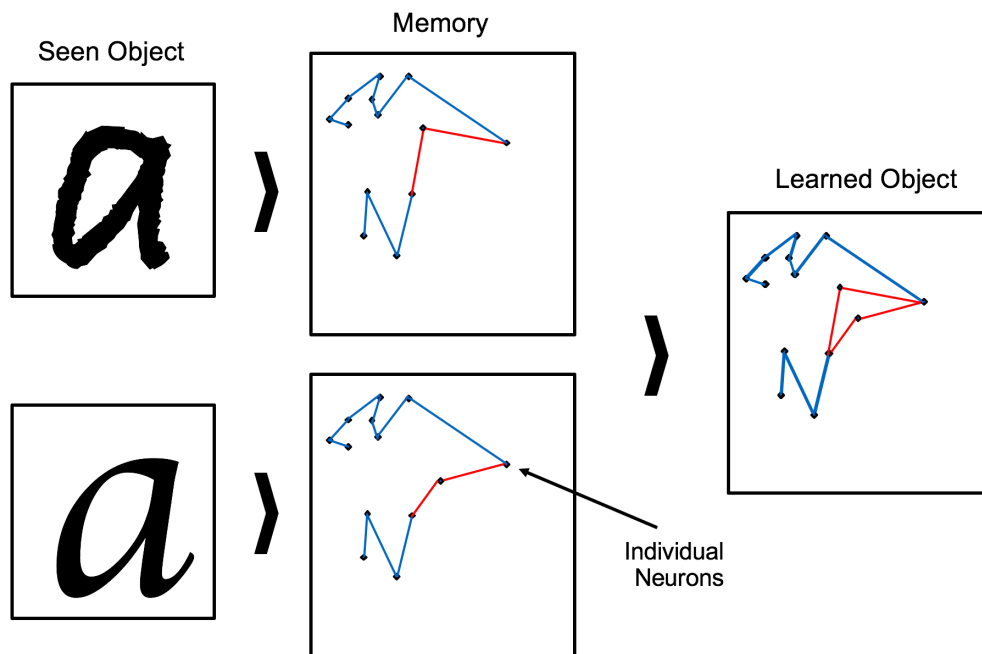


Figure 3.2. Conceptual view of how multiple memory formations can join to present same object.

3.4 Memory Recall Process

The recall process involves the output processing unit. After the memorization process has taken place and different variations of an object have been stored in the long-term memory, the stimulation of a version of the trained object will lead to activation of the memory cells associated with all close variations of that same object. At this point, memory cell activations generate stimulation in the OPU neurons, which ultimately leads to a physical manifestation. For example, seeing the character “a” in one font can activate neurons related to previously seen versions of “a” in other fonts, which in turn triggers the OPU associated with UTF-8 output to tell the operating system that the letter “a” was seen, as shown in Figure 3.3.

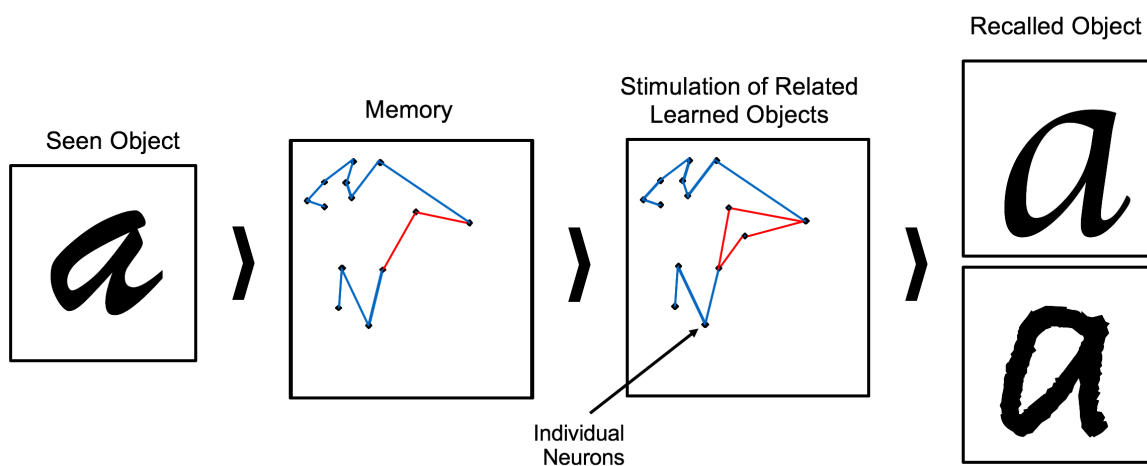


Figure 3.3. Conceptual view of how memory recall can occur, assuming that artificial brain has been previously trained on two different forms of letter “a”, thereby forming two cell assemblies with a large number of neurons in common, as shown in Figure 3.2; exposure of a new form of letter “a” results in commonality of neurons between new cell assembly and the other two, thereby activating them and triggering memory recall.

3.5 Associative Learning

Associative learning is responsible for the creation of event associations in our brain. In the late nineteenth century, a famous experiment was performed by a well-known

Russian psychologist named Ivan Pavlov, where a dog was repeatedly exposed to the sound of a bell when offered a piece of meat. Initially, the dog’s response to being shown the meat without the bells sound was salivation, but after repeated co-occurrences of showing the meat and ringing the bell, eventually the dog made an association between the bell ringing and the meat, whereby ringing the bell alone became the trigger for the dog’s salivation [32]. Implementation of associative learning in FEAGI was inspired by this behavior in the biological brain as follows: When an IPU processes a stimulus, say IPU A, this generates neuronal activities in its corresponding cortical pathway, ultimately activating a group of neurons in the memory region associated with the IPU; at the same approximate time, when IPU B processes a second stimulus, this leads to another unique set of neuronal activities in the memory region tied to IPU B. This is where we leverage Hebb’s hypothesis one more time and wire together the neurons from different memory regions that are firing together, as shown in Figure 3.4 and outlined by Algorithm 3.1.

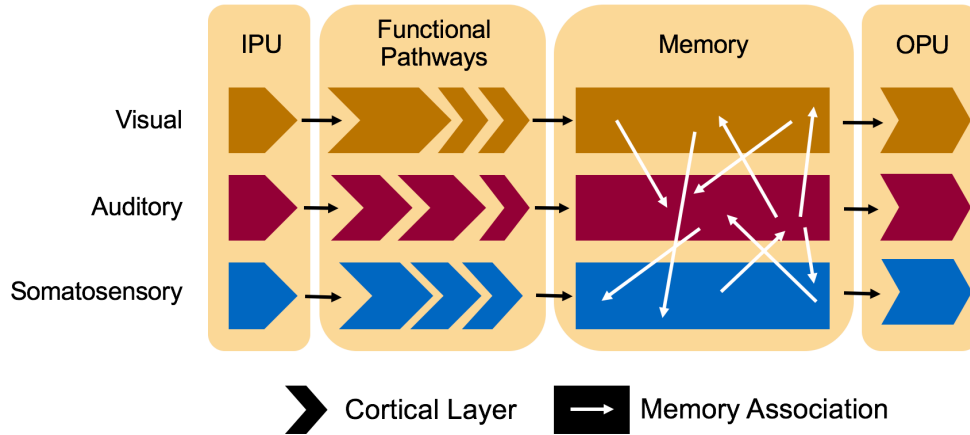


Figure 3.4. Conceptual view of the flow of sensory information within FEAGI. An IPU is dedicated to each input type. IPU’s output is directed to functional pathways that differ in structure and number of layers from one type to another due to the differences like raw information under process. The end of each functional pathway leads to a memory region dedicated to it. Memory association occurs when events happen in close time proximity across memory regions regardless of type, enabling the framework to generate an OPU signal in one sensory type, even though the input stimuli was exposed to another.

Algorithm 3.1 Memory Formation

```
1: procedure FORMMEMORIES
2:   for vision_memory_neuron in previous_fire_candidate_list do
3:     for destination_neuron in current_fire_candidate_list do
4:       if destination_neuron in vision_memory then
5:         apply_longterm_potential(vision_memory_neuron,
          destination_neuron)
6:       if destination_neuron in UTF_memory then
7:         apply_longterm_potential(vision_memory_neuron,
          destination_neuron)
```

Memory formation is a recursive and CPU extensive process, as shown in Figure 3.5. For each active neuron within the vision memory region, synaptic connectivities within the vision memory as well as synaptic connectivities to the other memory regions must be processed so the connection strengths among them can be updated accordingly. There are cases where a single cell assembly in the vision memory could become wired to multiple neurons in the UTF region. In such situation, inhibitory action is induced against connections to suppress non-singular connections between vision and UTF memory neurons.

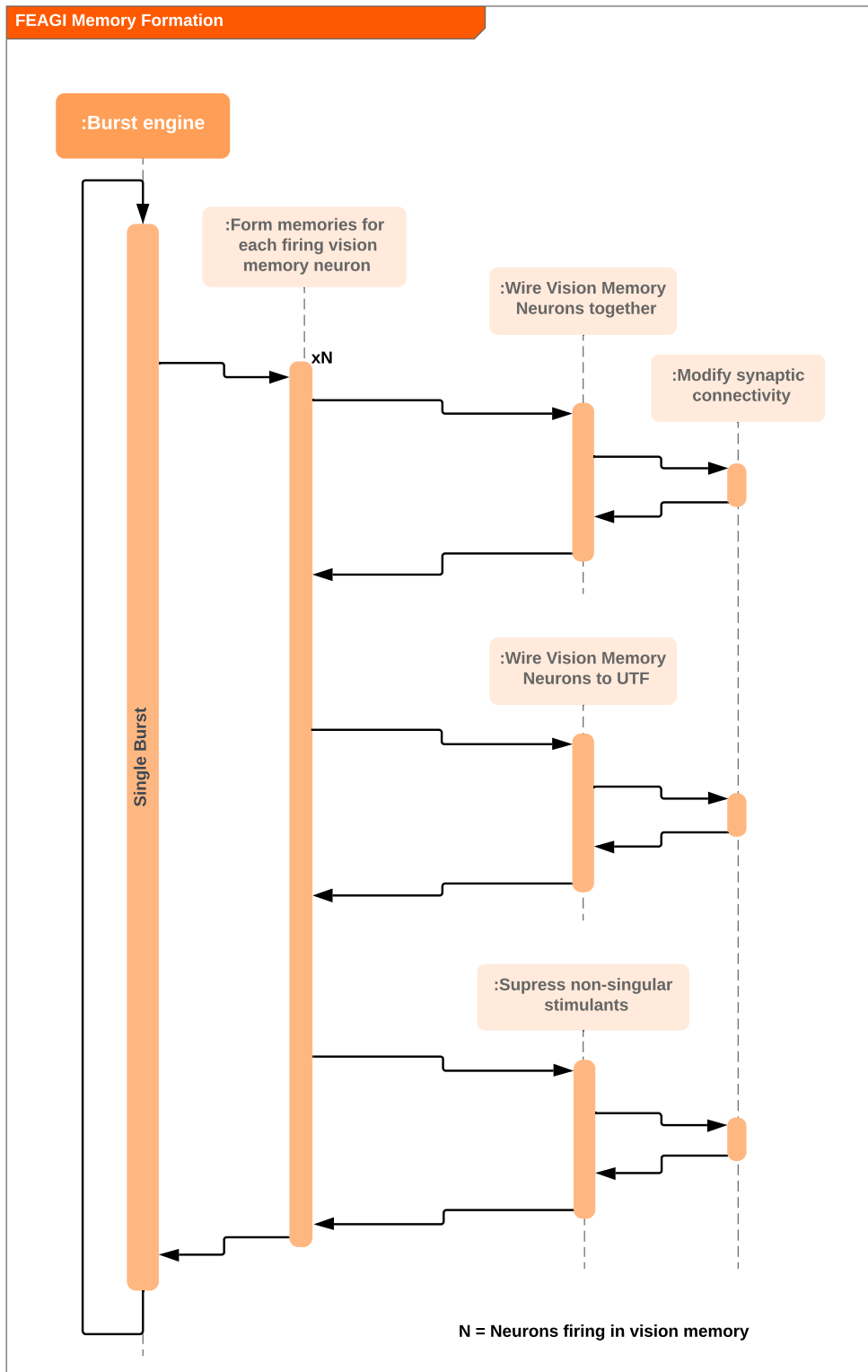


Figure 3.5. Sequence diagram for memory formation.

CHAPTER 4

TRAINING AND TESTING

4.1 Self-Learning

To be able to have the artificial brain evolve through generations, we saw the need to have a self-learning module so it can teach itself numbers and do a self-assessment at the end. Self-assessment results are captured in a database and used for fitness evaluations. For example, to be able to learn the number seven, the biological brain needs to be exposed to variations of this number over and over in a short period so that association can occur. To perform this operation In FEAGI, different variations of the character's image from the MNIST database are repeatedly fed to the vision IPU approximately when the correct UTF-8 version of it is fed to the UTF-8 IPU, as shown in Figure 4.1. This task is repeated over and over with multiple variations of the same character's image. Information flows in the form of neuronal activities through vision and UTF-8 cortical pathways simultaneously, and when reaching the corresponding memory modules, distinct cell assemblies begin to form. After the learning process for a particular character is complete, the self-learning algorithm moves on to the next character in the queue. This method was inspired by how children learn the alphabet through simultaneous printing and observation that is repeated many times.

4.2 Training Methods

Both supervised, and reinforcement learning techniques have been used as part of this framework.

- Supervised learning: Supervised learning is implemented in such a way as to read digits from the MNIST selection randomly and expose them to the IPU's for the exposure period. This is followed by a period of delay and repeat.

	Training						Testing						
Burst sequence	n	n+1	n+2	n+3	n+4	n+5	...	m	m+1	m+2	m+3	m+4	
Visual stimulus		5						5					
UTF Stimulus					5								
V1 activation													
V2 activation													
IT activation													
Vision memory													
UTF memory													
UTF output													
Comprehended UTF value												5	

Figure 4.1. Demonstration of how neuronal activities traverse through cortical layers during training and testing. Shaded blocks are an indicator of neuronal activity in the associated cortical layer (row) and burst instance (column).

- Reading from MNIST: Variations of labeled images of numbers from the MNIST database are read, and the image data along with label information are simultaneously fed to the vision and UTF input processing units correspondingly.
- Exposure duration: A variable has been defined to be able to adjust the amount of exposure each input would have to the system and is measured with the unit of burst count. The smallest exposure duration is a single burst. The exposure equates to feeding the raw data associated with the stimulus to the input processing unit repeatedly for the exposure duration. We have explored the impact of changing the exposure duration on trainability and noticed that lowering the exposure period below ten burst instances would have a negative impact, but increasing it above that threshold would not have any significant influence. We have not explored what parameters would drive this threshold, but we suspect that the number of neuron layers that the information needs to pass through before stimulating the output processing neurons would have an impact on it.

- Training order: The training module is designed to randomly read digits from the MNIST collection without any particular order, although we have experimented with training numbers sequentially –training 0 followed by 1, followed by 2, and so on– and noticed that random exposure has generally led to better results but not in a consistent manner.
- No activity period: It is essential to have a no-activity period between each exposure phase to assist with neuronal activities from the previous exposure decay within the cortical layers and help get rid of the noise before a new set of activities occurs.
- Reinforcement learning: We have leveraged the concept of pain and inhibitory neurotransmitters to perform reinforcement learning. Stimulation of a wrong image label leads to the activation of pain receptors. Pain receptors stimulate pain neurons, which in turn apply long-term depression (LTD) against the synapses involved in activation of a wrong response, as shown in Figure 4.2. Similarly, we award the synapses involved with the correctly recognized object by applying long-term potentiation (LTP).

4.3 Testing

The testing framework has much overlap with the training framework with the difference being that during testing, the label information is not fed to the system. After an image is exposed and the neuronal stimulations have passed through the cortical layers, a listener monitors the activities of the UTF8 output processing unit and records the results. There will be three scenarios: first, the stimulation of the output neuron could correspond to the correct image label; second, it could correspond to a wrong label; and finally, there could be no stimulation. We capture this information in a data structure and use it for the

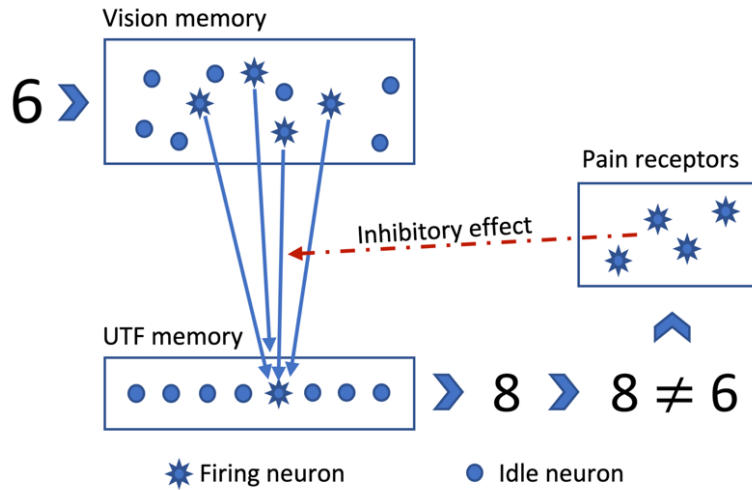


Figure 4.2. Concept of punishment as part of reinforcement learning implementation. In this example, number 6 is exposed to the system, but number 8 was detected. Due to the mismatch between the perceived number and the input number, pain receptors are stimulated to degrade the synaptic weights involved in producing the incorrect answer.

brain fitness calculation when the testing period is over. Ultimately, test results are saved in the genome database alongside the genome.

CHAPTER 5

EVOLUTIONARY MODULE

Multiple factors influence the evolution of a system, including imposed environmental constraints, the effects of external and internal environmental changes on the system dynamics, and survival needs. A system can evolve both while it is active and after becoming deactivated through genetic evolution.

A brain developed by FEAGI evolves from multiple aspects. The first evolutionary instance occurs during the artificial neuroembryogenesis process while the artificial brain is grown from a genome until it reaches maturity. The second occurs in the form of plasticity while the artificial brain is exposed to an external stimulus and undergoes the learning process. In this case, new connections are made, and existing connections are strengthened, deteriorated, or eliminated. The final evolutionary aspect of FEAGI is centered around using genetic algorithms to evolve the genome over generations, as explained below.

5.1 Genome Database

To improve an artificial brain instance, we collect statistical data associated with the overall artificial brain performance during cognitive tasks throughout its lifetime, and we store it in a genome database, alongside the genome responsible for the creation of that artificial brain, as shown in Figure 5.1. Collected statistical data are crucial to the creation of the future artificial brain offspring through genetic algorithms. As part of the offspring creation, mutation is injected into the genome so that the next generation can be created with a new network structure, thereby leading to a different performance. Genomes from high performing artificial brains are crossbred to generate a new generation.

As an example, as part of gene modification, the gene outlining the neuron density in vision V1 can be increased in value, which would result in the creation of a new arti-

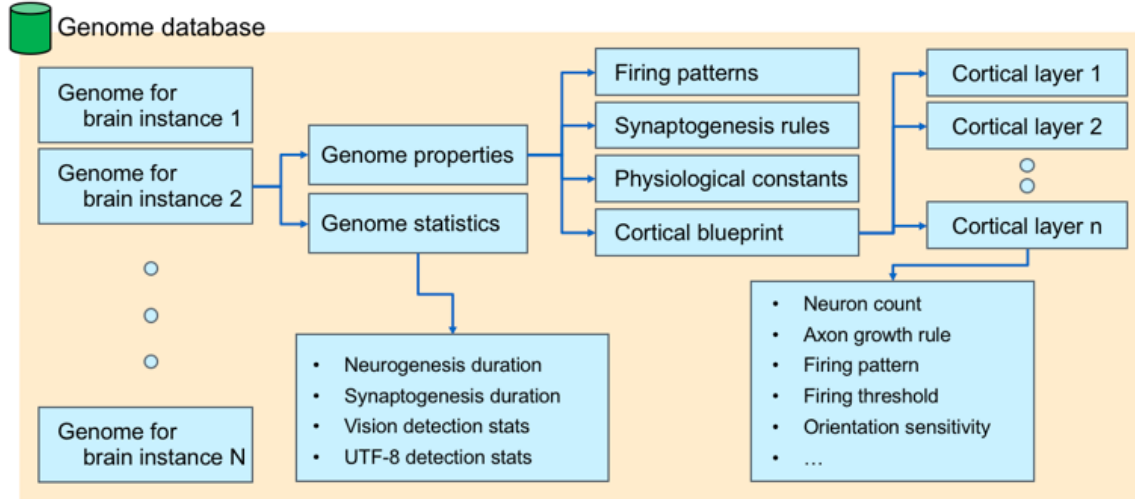


Figure 5.1. Structure of genome database.

cial brain with the V1 layer containing more neurons with different behavior. Similarly, different firing patterns can be exposed to different layers.

One of the deficiencies of the current evolutionary module is its inability to evolve the functions representing the biological brain’s physiology. To partially avoid this limitation, we have encoded constants associated with functions driving the artificial brain’s physiology within the genome so that at least the parameters can evolve. Part of the aim of our future research will be to assess how every single aspect of the human brain can be derived using the genome so that it can evolve in its entirety.

Our framework is capable of collecting statistical data while the artificial brain is interacting with external stimuli and save it in the form of metadata along with the genome. The burst management design, as discussed in Section 2.6, enables our framework to efficiently collect statistical information from all cortical areas while the artificial brain is active. Another important source of information for the evolutionary module is from the self-learning infrastructure. We measure the fitness of the system based on the percentage of correct MNIST digits recognized by the system after it has gone through the learning process.

It is worth discussing the design structure of our genome, which plays a key role in evolving the phenotype, the artificial brain. Every time our artificial brain goes through a life cycle, it collects statistical data, and when it is ready to shut down, all genomic information along with statistical data is stored back in the genome database. When a new offspring is generated, a new genome entry is added to the database. All we need to produce the next generation of an artificial brain is a genome from the database, along with a genetic algorithm.

Figure 5.1 outlines the hierarchy involved in the genome database. For each genome instance, there are two main sets of information captured: one set, categorized as “properties,” captures all parameters needed to regrow a new artificial brain; the second set, categorized as “statistics,” captures the artificial brain’s performance and behavior during its lifetime.

5.2 Genetic Algorithms

The core premise of FEAGI as an evolutionary framework is to allow the artificial brain’s architecture to evolve through generations. To this end, we have developed an evolutionary Python library as part of FEAGI, capable of reading a genome instance and performing genetic algorithms as follows:

- Selection: Every time a new brain instance is about to be developed, the first step is to provide the genome. We have introduced a logic to randomly select from the following three options for genome selection:
 - Select a genome from the database with the highest evaluated fitness.
 - Select the newest genome in the database.
 - Randomly select any genome from the database.

After the selection, we randomly decide on whether to use the selected genome as is or to let it go through either mutation or cross-over.

- **Mutation:** Mutation is performed by modifying the value of various parameters (genes) within the genome by a random percentage between -30% and $+30\%$. The choice of $\pm 30\%$ was initially a good choice because it allowed the genetic algorithm to explore a broader range of parameter values without getting stuck in a local minima. After the fitness grew beyond 50%, we changed the window to $\pm 10\%$ to allow for smaller fluctuations and hence finer tuning. Changing parameter values through mutation would result in a new anatomical structure and influences the behavior of the functions simulating brain physiology that ultimately leads to an improved or diminished system performance. In our current implementation, the mutation is limited to a subset of genes within the genome, meaning that not all parameters defined within the genome are subject to mutation. The reason for this choice is to limit initial development efforts, indeed an area for future improvements.
- **Crossover:** Crossover occurs when a portion of one genome is swapped with another. In our implementation, we have adopted a simplified approach. We randomly select two genomes from the pool of top-performing genomes available in the database, and then read a random set of genes from one genome and swap values with its counterpart.

CHAPTER 6

FRAMEWORK HIGHLIGHTS

FEAGI consists of two significant workflows that closely interact with each other: one is the evolutionary workflow that handles genome and is responsible for regenerative tasks, and the other, the information workflow that is responsible for interacting with outside world, data-processing, memorization, and learning, as shown in Figure 6.1.

Two of the essential characteristics of FEAGI as a brain simulation framework are *scalability* and *evolvability*. Considering the sheer number of neurons working together in the human brain to achieve cognitive tasks, it is vital to expect the same from a framework attempting to replicate its behavior. The scalability and evolvability factors become crucial when our goal is to achieve an efficient and effective network design and architecture. Here, we list some of the contributing factors in scalability and evolvability of the framework. The framework offers the user two options during the initial launch: one to grow an artificial brain from scratch using the genome file and the second to load a previously saved one. The latter option would load a fully connected artificial brain along with all the memories and learned artifacts.

6.1 Scalability

- Modularity of genome structure
 - Genome modularity can enable any number of cortical areas to be defined independently and added to an existing artificial brain model.
 - The genome is structured such that scaling up the artificial brain by adding cortical layers would be as easy as adding a few entries in genome that would define the cortical layer's anatomical properties.

- Genomes are stored in MongoDB database in the form of JSON documents. MongoDB is extensible and highly scalable, enabling FEAGI to capture genomic information for millions of generations.
- Burst engine
 - The burst engine is inspired by the sparsity of neurons firing in the biological brain. Our implementation limits the CPU and memory usage to only neurons in the burst engine fire queue. This technique is a crucial scalability factor leading to a reduced memory and energy footprint needed to operate an artificial brain with billions of neurons.
- Connectome design and structure
 - We have chosen an object-oriented data structure to capture the artificial brain's anatomy in the connectome so that it can be stored in a scalable database format. As of now, and without any compression or optimization, each neuron requires about 1 kilobytes of storage. Size of the connectome scales linearly with the number of neurons. If an entire human brain with about 100 billion neurons is simulated, then would require 100 terabytes of storage with this implementation, which is well within the supported limits of existing document databases such as MongoDB.
- Multiprocessing
 - Even though the current implementation is CPU based, we have leveraged multiprocessing techniques to allow parallel operations, such as neuron firing, to take advantage of multiple CPU cores. It is part of our future roadmap to extend the framework to leverage the GPU and, most importantly, neuromorphic hardware.
- Modularity of input and output processing units

- Each input or output processing unit is independent of the others and can plug into the framework by passing along processed stimuli to the cortical pathways. This design enables the framework to add support for processing new types of input such as sound or tactile sensation as well as interacting with the outside world through various output methods.

6.2 Evolvability

- Genome-based growth
 - Our design is based on segregating brain anatomy from its physiology. The genome contains enough information to be able to grow an artificial brain step by step, thereby enabling an evolutionary algorithm to be used to influence its growth and maturation, which can ultimately be used to improve its performance over generations.
 - The genome database enables data analytics to be performed against statistical data collected from each running brain, which can be a powerful tool in identifying influential genes and high-performing genomes.
- Plasticity
 - Synaptic plasticity evolves the artificial brain structure, thereby enabling learning and memorization.
- Scale-out evolution
 - Our framework has the capability of spawning multiple artificial brain instances simultaneously in order to speed up going through multiple generations as part of the evolutionary process. There is currently a limitation that multiple instances can be created only within a single operating system instance, and it is our goal to lift this limitation in future iterations.

6.3 Insights

The artificial brain built by FEAGI, similar to the biological one, can grow considerably large and encompass numerous neurons and synaptic connections. To gain insight into the interworking of the artificial brain, we designed and implemented an infrastructure to allow raw data to be collected while the brain is active and have them stored in a time-series database called "InfluxDb." We then leveraged an open-source monitoring solution called "Grafana" to develop a custom dashboard to help visualize activities in various regions and cortical areas, as shown in Figure 6.2.

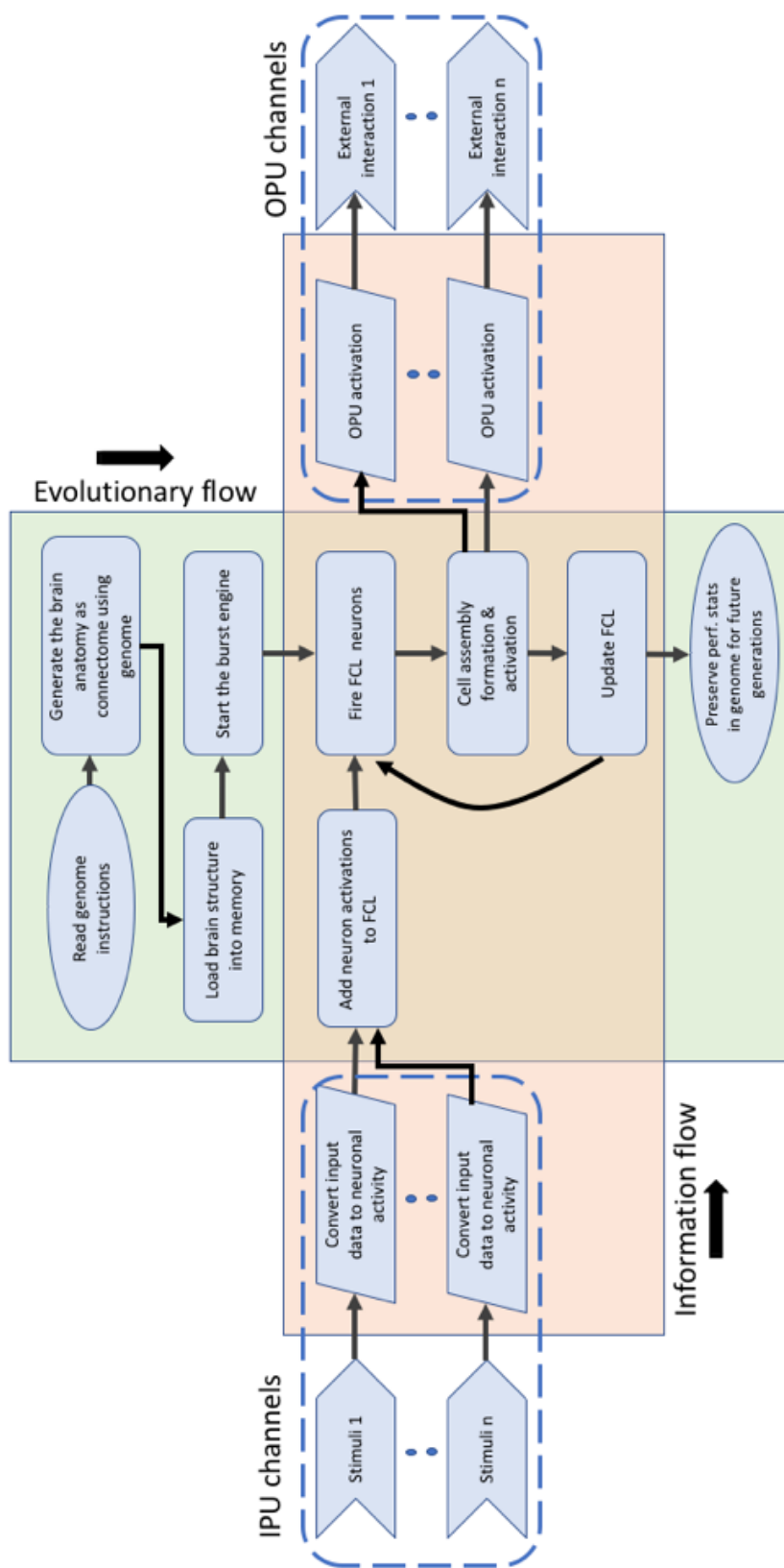


Figure 6.1. FEAGI system-level workflow.

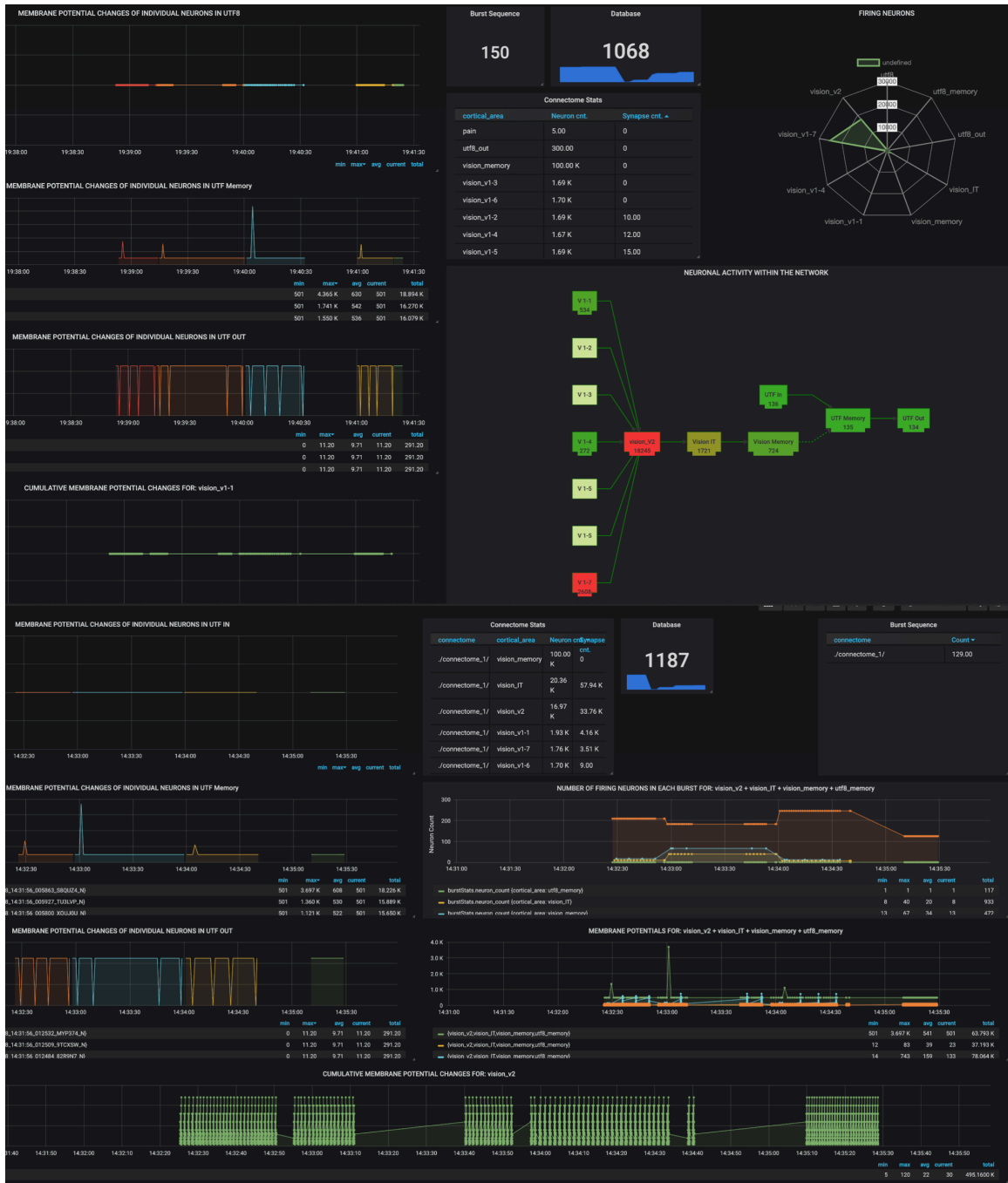


Figure 6.2. Monitoring dashboard created to help monitor cortical activities across various regions.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 Proof of Concept

To demonstrate the feasibility and success of this framework, we have implemented a simple simulation of the visual system that is limited to monovision in grayscale, which is only capable of processing static images. We have developed two different IPUs; one capable of reading 28x28 images from the MNIST database, and the other capable of reading UTF-8 characters from standard input. UTF-8 input has been used for supervised learning to teach the artificial brain the association between a character's image and its UTF-8 version.

The visual cortical pathways in our proof of concept have been built after the human brain's ventral visual pathway consisting of layers V1, V2, V4, and IT, but for simplicity, we have omitted layer V4. Similar to the human primary visual cortex, we have defined orientation selectivity for V1 sublayers. Layer V1 in the human brain consists of six sublayers, and we have implemented the same. Additionally, in layer V1 of the biological brain, there is the concept of ocular dominance columns [32], which we have ignored since our focus is only on monovision implementation. We have also ignored structures such as cytochrome oxidase blobs [32] since they relate to color detection, and our implementation has been limited to grayscale. From the connectivity perspective, vision IPU output is mapped to the six V1 sublayers. Every V1 sublayer projects its axons to layer V2, and layer V2 has its neuron axons projected to layer IT. Individual neuron connections within layers as well as from one layer to another are driven by the synaptogenesis rules that we have defined in the genome. In our implementation, we tried to define the genome parameters in a way that is close to our understanding of the human brain.

Memory neurons in the genome are defined in a way so as not to have synaptic connectivity to others initially but rather programmed so that upon simultaneous firing,

long-term potentiation can occur, and they synapse each other. As a group of memory neurons becomes activated over and over at approximately the same time, they bond and form the cell assembly, as described in Section 3.1, and help to shape memories.

Visual memories are created by the formation of cell-assemblies that collectively represent the memory of a particular object. We have visualized the formation of cell assemblies as the artificial brain undergoes training, as shown in Figure 7.1. As depicted, each color is representative of a cell assembly associated with a particular number. By looking at this image, one can observe that each cell assembly has a unique shape and form. Another observation is the fact that there are many overlaps between various cell assemblies. This overlap harms the fitness of the brain because it leads to misidentification of one number with another.

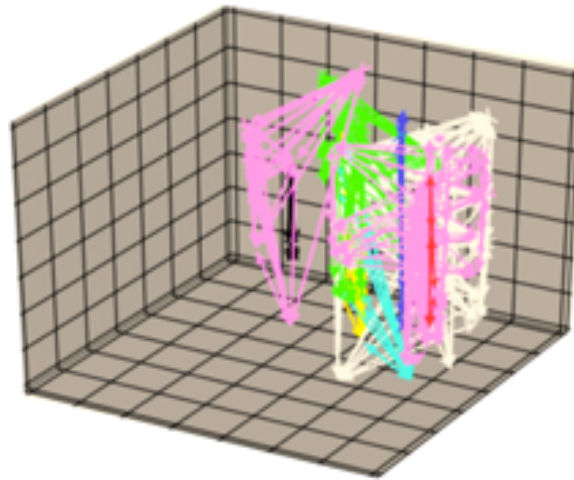


Figure 7.1. Visualization of cell assemblies. Each color cluster corresponds to the cell assembly associated with a particular number.

We have developed an OPU capable of translating UTF-8 memory patterns back into a UTF-8 character. Development of the UTF-8 OPU has enabled us to build an end-to-end example of how information can be fed to the artificial brain, stored in long-term memory, and presented back to the outside world within the proposed framework.

The UTF-8 IPU and OPU have a simple cortical structure and are custom made to be the translation layer between the neuron spikes and the operating system. The UTF-8 IPU, OPU, and memory are all one-dimensional cortical regions consisting of a single column of neurons, where each neuron's axon is connected directly to its corresponding neuron in the downstream UTF-8 layer, as shown previously in Figure 2.7.

The process of image recognition starts with the vision IPU, (refer to Section 2.5 for more details) and continues through the cortical pathways. By passing through the vision IPU, the image breaks down into its fundamental components and feeds as a series of spikes to the first layer of the cortical pathway connected to the vision IPU, which happens to be layer V1. Neurons in each cortical layer have axons connected to other neurons as part of their subsequent layers, so activation in one layer propagates through the cortical pathway and eventually reaches the final layer, or IT layer. This layer is the only one with synapses to the memory neurons, so when neurons in layer IT are activated, they stimulate memory neurons associated with visual memory. When the act of seeing the image is repeated over and over, this leads to the stimulation of the same memory neuron ensemble over and over, thus leading to the formation of long-term memory.

The next phase of learning is the association of a UTF8 character with the visual memory. The essential factor in enabling the artificial brain to learn and associate the visual representation of characters with the UTF-8 version is the wiring of neurons from the visual memory to the UTF-8 memory. When neurons from both memory regions fire together, they wire together and improve their synaptic connectivity. UTF8 has its dedicated cortical pathway, which is simple. When the user enters a key on the keyboard at the same approximate time the image is read by the vision IPU, the UTF signal propagates and reaches its UTF8 memory region. At this point, we leverage the long-term potentiation technique again but now between the two memory regions. As the cell assemblies in both memory regions become activated, they become wired together, so now the activation of one can lead to the activation of the other. This simple yet powerful function

would tie memories from one functional memory region to another, so that the simulation of one leads to the stimulation of the other.

Figure 7.2 depicts the neuronal activities in our framework when the number 4 is read from the MNIST database and passed through visual pathways. The activation of neurons in the IPU creates a chain reaction leading to the activation of neurons throughout the cortical pathway. On the other hand, at the same time frame when the number 4 is read from the MNIST, the user types the character 4 in the framework, it is fed to the UTF-8 IPU, and from there it passes through the UTF-8 cortical pathway. Every time the neurons that collectively represent the number 4 from the vision and UTF-8 memory regions fire, their connectivity improves, and eventually the activation of one group can lead to the activation of the other. Similar to a biological brain, the learning process demands repetition and proper timing. When we learn a new object, we look at it and repeat its name over and over to help memorize the association. The same happens in our framework. After multiple instances of simultaneous memory stimulations, the artificial brain learns that an MNIST version of number 4 is associated with the UTF-8 version of 4.

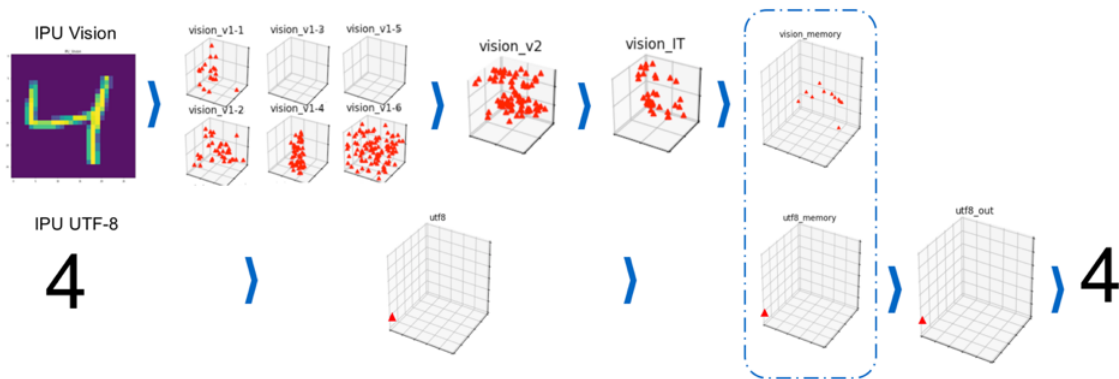


Figure 7.2. View of neuronal activities within the framework leading to association of UTF-8 version of character with handwritten version read from MNIST database.

During training, the artificial brain is exposed to both image and UTF representation of a character at the same time, so memory associations occur, and later on, when the

same character is seen again, this leads to the activation of the same cell assembly and consequently the activation of its UTF counterpart in another memory region. At this point, the neuron connections of the UTF8 OPU with its memory neurons help stimulate the OPU region associated with the seen character, thereby resulting in the display of character on the computer display.

When a new object exposed to the system is similar to a previously seen version, this leads to the stimulation of a large majority of neurons representing the original object. This stimulation fuses the cell assembly of one object to the other, and if it happens that the first object was associated with a UTF-8 character, then the new object will stimulate the same character as well. Figure 7.3 depicts this phenomenon. First, the artificial brain is trained to recognize variations of the number 1 along with the UTF-8 version of it. Next, the artificial brain is exposed to an unseen version of the number 1, which leads to the stimulation of the UTF-8 version of the number 1 as well.

7.2 Results and Discussion

We have leveraged our proof of concept implementation and allowed it to evolve through many generations. An area of interest was to prove that the artificial brain created by FEAGI has the capability of improving itself over generations. To evaluate improvements, we have defined a fitness function as follows:

$$Fitness = \frac{1}{1 + e^{(-t+a)}} \times \frac{c}{t}, \quad (7.1)$$

where c is the number of times the artificial brain has been capable of correctly identifying a stimulus, t is the total number of times the stimulus has been exposed to the brain, and a is the activity threshold. In the fitness formula, the first fraction is a shifted sigmoid function that imposes a negative influence on the function when the number of exposures is below a threshold.

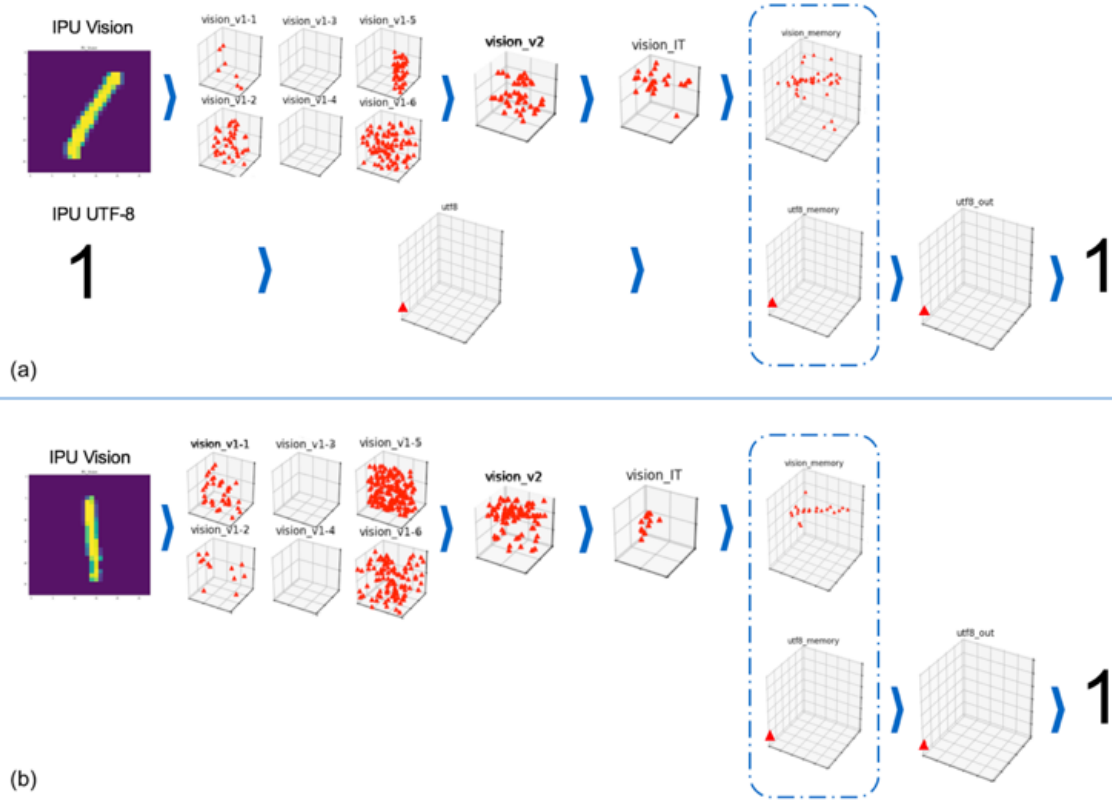


Figure 7.3. Framework behavior during learning and recall: (a) exposure of handwritten version of number 1 along with UTF-8 version of number 1 to framework at same approximate time, leading to association of cell assemblies between visual memory and UTF-8 memory; (b) exposure of framework with another handwritten version of number 1 leads to activation of same cell assembly in UTF-8 memory region and recognition of character 1 in association with handwritten version.

7.2.1 Performance Improvements over Time

We have developed a baseline genome outlining the growth rules, anatomical properties, and limited physiological characteristics of various cortical regions, and we have set the brain to evolve over thousands of generations using two sets of MNIST digits from 0 to 9. Each generation has gone through the self-learning and self-assessment processes, and as a result, a fitness value was calculated and associated with a given genome. As shown in Figure 7.4, the fitness of the brain in the latest generations has improved significantly compared to the earlier ones, and the overall trend is positive. We can also observe an interesting phenomenon that is highlighted by the letters A, B, and C. These letters

are indicators of the generation points where we manually induced a physiological mutation in the system. We observe that the fitness trend noticeably changes at these points, especially point C. We hypothesize that the fitness improvements if only driven by the anatomical changes would reach a resistance level after finite generations and will only fluctuate within a range until a physiological change can take it to the next level, as shown by point C in Figure 7.4. We are deferring the proof of our hypothesis to future research. Point D in Figure 7.4 corresponds to the generation where a feature for early detection and termination of low performing brains has been introduced. This feature monitors the level of neuronal activities in the visual memory, and if the level is zero, proactively terminates the training process and eliminates that generation. This feature has enabled us to have a more efficient genetic algorithm by not wasting time going through the full training and testing process on generations with low potential. As a result, we observe a relatively empty region right after point D, highlighted with a dotted line oval. Point E in the Figure is associated with the introduction of reinforcement learning capability as explained in Section 4.2.

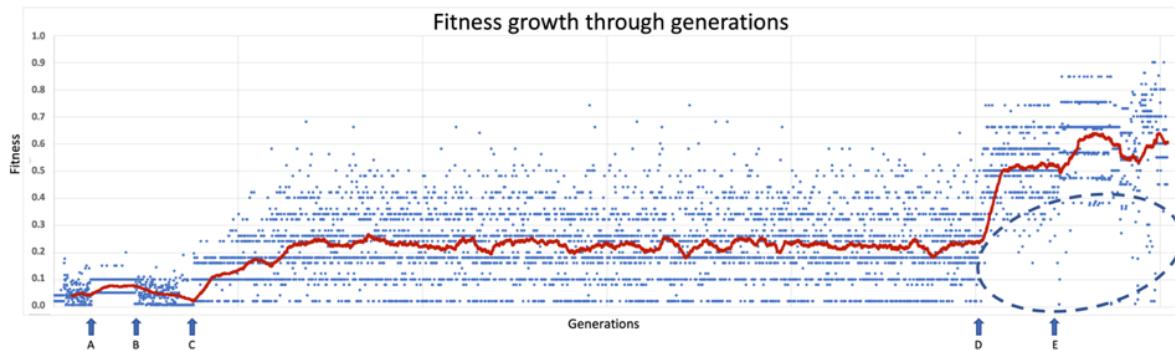


Figure 7.4. Artificial brain's fitness improvement over generations. Individual dots outline the fitness value for a given generation; the line is a moving average over the past 100 data points; letters A, B, and C are indicators of a generation phase where a manually induced physiological mutation has occurred; letter D is where we introduced the early termination feature; letter E is where reinforcement learning was introduced; and the dotted line oval is highlighting the observation of less low-performing instances due to the implementation of the early termination feature. Data points with a fitness of zero have been excluded from the graph.

An important factor to discuss is the fitness value. As shown in Figure 7.4, our current measurement of fitness ranges from 0 to 0.9, with 1 being the perfect score. We acknowledge that this is a low score in comparison to state-of-the-art handwriting-recognition algorithms [43], and for our initial proof of concept, we have identified below a few areas that we think are contributing to not achieving a competitive level of fitness:

- **Evolutionary constraints:** In the biological brain, physiological activities play a crucial role in how the brain functions. In our implementation, we have simulated such activities by writing functions in Python; an example is the one for neuron firing. As part of these functions, variables have been defined that influence how the function behaves; an example is a variable representing the firing threshold for the neuron firing function. We have externalized all such variables used as part of functions to the genome so they can evolve. The major shortcoming that still stands is the ability to evolve the structure of the code defining the functions themselves. In this regard, we propose two areas for future research: first, to find methods and approaches to limit code-dependency for the physiological functions, and second, to investigate a framework with self-evolving code capability.
- **Constraints on parallelism:** Our code is currently only leveraging the CPU and is incapable of tapping into the GPU resources. Leveraging the GPU can help reduce the training time considerably, thereby leading to a shorter lifecycle for each brain instance and ultimately a higher number of generations over time. Taking advantage of the neuromorphic hardware would take this one step further.
- **Scale-out limitations:** Currently, we cannot scale the creation of brain instances beyond a single operating system instance, which is limiting us to have the framework run on multiple servers while sharing the same genome database. Adding this feature would allow the system to infinitely scale-out and leverage a large number of

compute nodes in the cloud or within a data center to help simulate a much larger population of evolving artificial brains in parallel.

7.2.2 Comprehension in Real-Time

Even though the error rate in our trained model is still quite high and would require many more generations to become competitive, we have a unique advantage, and that is the comprehension speed. The time it takes for the artificial brain developed by FEAGI to detect an exposed image is currently around $160ms$ while running on a laptop, which can be considered as real-time. In contrast, Spaun [7], the leading brain simulator based on spiking neurons, would require around 2.5 hours of processing to simulate one second of handwriting recognition task. The reason for such a drastic difference goes back to our design philosophy: to be inspired by the brain but not to imitate it. In our design, we have tried to simplify the model and rely on evolution to help us achieve a functional architecture suitable for a digital environment. As an example, our decision to substitute thousands of synapses between two neurons with a single synapse averaging the post-synaptic influence has resulted in considerable simplification and significant performance gain. Similarly, we have simplified physiological behaviors in a way to simulate the intended outcome without imitating all the steps followed in a biological counterpart. Spaun has adopted a lower-level architecture which on the positive side would be a very close representation of the human brain and can help neuroscientists study and emulate brain behaviors, but it comes at the cost of complexity requiring extensive processing power for simulation that prevents it from operating in real-time on commodity hardware.

7.2.3 Framework Versatility

As part of our proof of concept, we went through more than 25,000 generations, with fitness being evaluated using the MNIST handwriting dataset. After we reached the fitness of 0.90, we started to train the new generations using the Fashion-MNIST instead.

Initially, the fitness for apparel recognition was less than 0.1, but in less than 100 generations, it rose to 0.65 but stayed at that maximum level over future generations. We do not have any firm explanation for this behavior, but our theory is that the fitness rose very quickly because the seed genome used for apparel training was previously evolved through thousands of generations for handwriting recognition, hence the artificial brain's structure was optimized for general image recognition. Since Fashion-MNIST contains categories with close similarities, such as T-shirts and pullovers, we suspect that our visual pathway in the early regions have not evolved enough to be able to distinguish certain features.

7.2.4 Brain Footprint

We have measured the amount of storage consumed by the connectome database in order to forecast the system requirements as brain scales. As shown in Figure 7.5, in our current implementation, a brain with 500,000 neurons consumes approximately 1.3 gigabytes of storage. We are currently storing brain anatomical information in JSON format without performing any data serialization or compression that could considerably reduce the footprint. An important observation in Figure 7.5 is that the growth rate is parabolic, because in our experiment, we have kept the size of each cortical area dimensions constant while increasing the neuron count. Keeping the dimensions constant leads to a denser cortical region with more neurons per unit of space. In this situation, there is a higher probability for neurons to synapse, thus leading to excess synapse creation, which is the cause of non-linear capacity growth.

7.2.5 Impact of Exposure Time on Fitness

Figure 7.6 in Appendix highlights the results of our research on how changes in stimulus exposure time could impact fitness in a particular generation. We can observe that a slight change in the exposure time can have a significant impact on the fitness value—changing the exposure time from 8 to 15 bursts can improve fitness up to 20%. We also

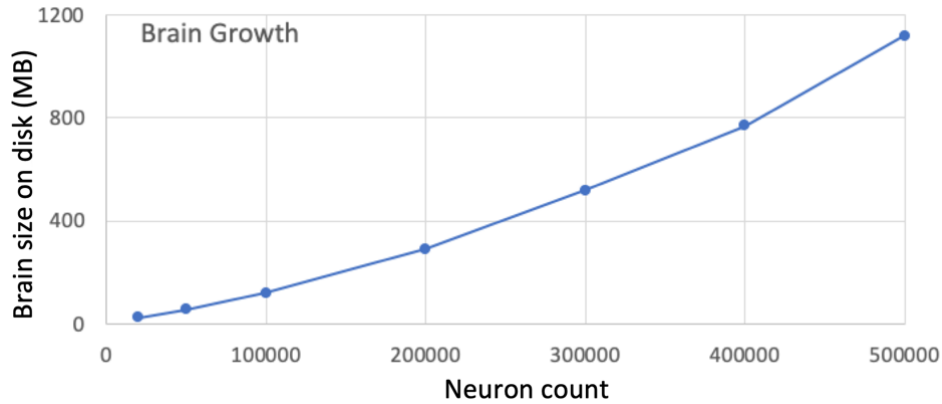


Figure 7.5. Relationship between total neuron count and disk space consumed by artificial brain.

observed that for this particular genome, increasing the number of training sets from 10 to 60 is not contributing to fitness improvements. After further investigations by monitoring cortical activities in the layers leading to the visual memory region as well as inspecting the cortical anatomical properties, we determined that the selected genome was mutated as such that has led to sub-optimal synaptic connectivity between layers V4 and IT. This deficiency led to the scarce and localized firing of neurons in the IT region that subsequently created a narrow data flow to the visual memory region, which ultimately caused low learning potential for the artificial brain despite the availability of additional training data. It should be noted, however, that the evolution of genes over time influences this behavior over generations, leading to improvements in fitness.

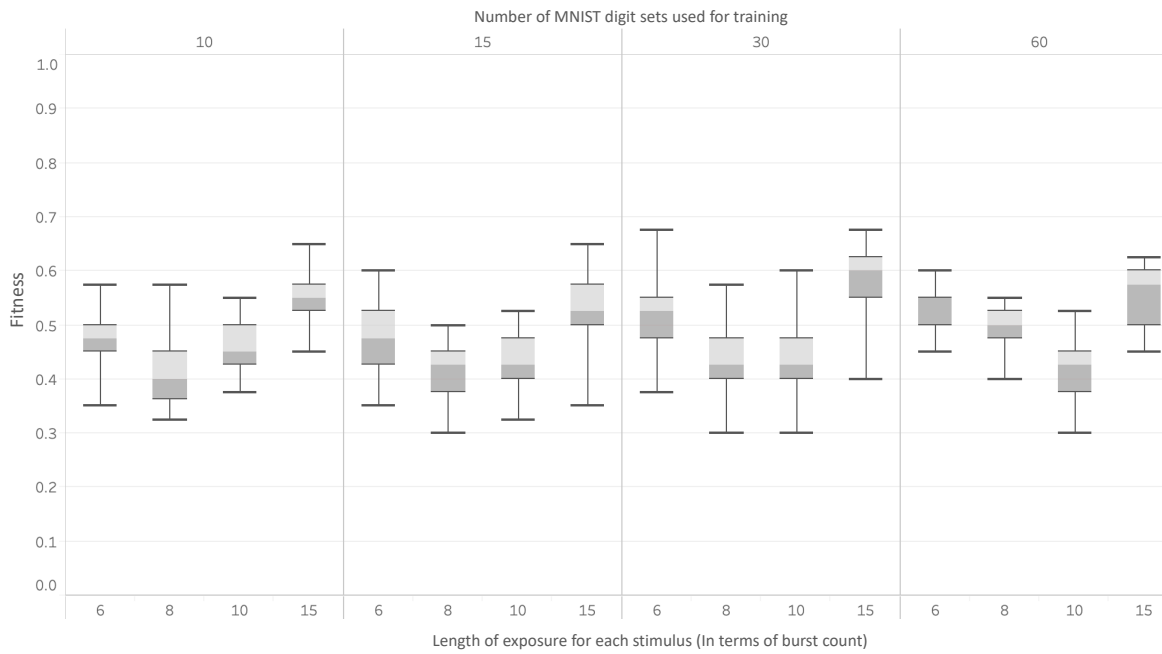


Figure 7.6. Impact of exposure time on fitness and its correlations with the size of training set. For this analysis, a random genome was used frequently with a different number of training sets and exposure times. Each training set is considered 10 MNIST digits, and exposure time is the duration by which every training or test sample is exposed to the input processing unit.

CHAPTER 8

CONCLUSION AND FUTURE WORK

8.1 Conclusion

Many existing brain simulation frameworks have cognitive abilities, accuracy, and visualization capabilities that are far superior to what we have offered here in our framework. The differentiating factors in our proposal are the evolvability and scalability enabled by our chosen architecture based on a novel encoding technique inspired by the neuroembryogenesis process in the human embryo. In our proof of concept, we demonstrated an artificial visual cortex generated from a genome simulating the ventral visual pathway of the human brain capable of learning, memorizing, and recalling digits in real-time read from the MNIST database. Nature has proven the power of evolution; however, over time, through natural selection, the biological brain has obtained its capabilities. One can reason that with the right approach, a synthetically developed brain could achieve the same.

8.2 Future Work

We see ourselves at the beginning of a long road of discovery and research ahead. However, we are confident that the chosen modular architecture with evolutionary capabilities would be able to provide a significant boost in accelerating our progress in creating an artificial brain with artificial general intelligence. Our current proof of concept demonstrates the framework’s functionality but also highlights the lack of accuracy needed for effective character recognition, especially compared to recent deep-learning models.

Many factors work hand in hand to constitute a functioning brain, such as the connectivity among neurons, cortical neuron density, and neuron firing properties. The genome primarily defines these factors. Currently, the genome must be tweaked to improve fitness, and we have leveraged genetic algorithms to help the brain evolve and improve.

Our current implementation requires further work to take advantage of genetic algorithms in a compelling way. One of the areas that require attention is the ability to evolve the brain's physiological properties. In our current implementation, we have parameterized just a few functions representing physiological activities, and future work is necessary to enable all brain aspects to evolve.

Another area for future research is to add additional input processing units, such as tactile and auditory, along with additional output processing units such as motor and speech. The definition of new input and output processing units would enable the development of new cortical pathways and higher-level cognitive functions such as translation of an auditory signal to a physical movement. The concept behind IPU is not novel and has been implemented as an interface to all current spiking neural network frameworks to convert stimuli to spike train[44]. Future work is to expand FEAGI's IPU capabilities by either defining new IPUs or to integrate the existing ones.

Alternatively, we are looking for ways to build an interface between data obtained by the Human Brain Project [5] and our framework so that we can automate the creation of a seed genome as the first generation of our framework instance. An option to approach this task would be to collect the cortical properties from the HBP data and have them translated to genes part of FEAGI's genome.

Last, leveraging neuromorphic hardware can have a significant impact on improving the artificial brain's performance. It is our goal to build extensions to our framework, thereby allowing integration with leading neuromorphic hardware such as IBM TrueNorth[14], NeuroGrid[12], and SpiNNaker[11].

REFERENCES

REFERENCES

- [1] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of go without human knowledge. *Nature*, 550(7676):354–359, 2017.
- [2] Diego Ardila, Atilla P Kiraly, Sujeeth Bharadwaj, Bokyoung Choi, Joshua J Reicher, Lily Peng, Daniel Tse, Mozziyar Etemadi, Wenxing Ye, Greg Corrado, et al. End-to-end lung cancer screening with three-dimensional deep learning on low-dose chest computed tomography. *Nature medicine*, 25(6):954–961, 2019.
- [3] Yann LeCun, Corinna Cortes, and CJ Burges. Mnist handwritten digit database. *AT&T Labs*, 2:18, 2010.
- [4] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *CoRR*, 2017.
- [5] Henry Markram. The human brain project. *Scientific American*, 306(6):50–55, 2012.
- [6] Chris Eliasmith and Oliver Trujillo. The use and abuse of large-scale brain models. *Current Opinion in Neurobiology*, 25:1–6, 2014.
- [7] Chris Eliasmith, Terrence C Stewart, Xuan Choo, Trevor Bekolay, Travis DeWolf, Yichuan Tang, and Daniel Rasmussen. A large-scale model of the functioning brain. *Science*, 338(6111):1202–1205, 2012.
- [8] Chris Eliasmith and Charles H Anderson. *Neural engineering: Computation, representation, and dynamics in neurobiological systems*. MIT Press, 2004.

- [9] Trevor Bekolay, James Bergstra, Eric Hunsberger, Travis DeWolf, Terrence C Stewart, Daniel Rasmussen, Xuan Choo, Aaron Voelker, and Chris Eliasmith. Nengo: A python tool for building large-scale functional brain models. *Frontiers in Neuroinformatics*, 7:48, 2014.
- [10] Wolfgang Maass. Networks of spiking neurons: The third generation of neural network models. *Neural Networks*, 10(9):1659–1671, 1997.
- [11] Andrew Mundy, James Knight, Terrence C Stewart, and Steve Furber. An efficient spinnaker implementation of the neural engineering framework. In *Proceedings of the IEEE International Joint Conference on Neural Networks (IJCNN)*, pages 1–8, Budapest, Hungary, July 2015.
- [12] Ben Varkey Benjamin, Peiran Gao, Emmett McQuinn, Swadesh Choudhary, Anand R Chandrasekaran, Jean-Marie Bussat, Rodrigo Alvarez-Icaza, John V Arthur, Paul A Merolla, and Kwabena Boahen. Neurogrid: A mixed-analog-digital multichip system for large-scale neural simulations. *Proceedings of the IEEE*, 102(5):699–716, 2014.
- [13] Rajagopal Ananthanarayanan, Steven K Esser, Horst D Simon, and Dharmendra S Modha. The cat is out of the bag: Cortical simulations with 109 neurons, 1013 synapses. In *Proceedings of the IEEE Conference on High Performance Computing Networking, Storage and Analysis*, pages 1–12, Portland, Oregon, November 2009.
- [14] Jun Sawada, Filipp Akopyan, Andrew S Cassidy, Brian Taba, Michael V Debole, Pallab Datta, Rodrigo Alvarez-Icaza, Arnon Amir, John V Arthur, Alexander Andreopoulos, et al. Truenorth ecosystem for brain-inspired computing: scalable systems, software, and applications. In *Proceedings of the IEEE International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 130–141, Salt Lake City, Utah, USA, November 2016.

- [15] Robert Preissl, Theodore M Wong, Pallab Datta, Myron Flickner, Raghavendra Singh, Steven K Esser, William P Risk, Horst D Simon, and Dharmendra S Modha. Compass: A scalable simulator for an architecture for cognitive computing. In *Proceedings of the IEEE International Conference on High Performance Computing, Networking, Storage and Analysis*, page 54, Salt Lake City, Utah, USA, November 2012.
- [16] Dany Varghese and Viju Shankar. Cognitive computing simulator-compass. In *Proceedings of the IEEE International Conference on Contemporary Computing and Informatics (IC3I)*, pages 682–687, Mysore, India, November 2014.
- [17] Marc-Oliver Gewaltig and Markus Diesmann. Nest (neural simulation tool). *Scholarpedia*, 2(4):1430, 2007.
- [18] Dan Goodman and Romain Brette. Brian: A simulator for spiking neural networks in python. *Frontiers in Neuroinformatics*, 2, 2008.
- [19] Philipp Koehn. Combining genetic algorithms and neural networks: The encoding problem. Master’s thesis, The University of Tennessee, Knoxville, 1994.
- [20] F Gruau. Cellular encoding of genetic neural networks, Technical report 92-21, Laboratoire de l’Informatique du Parallélisme. Ecole, 1992.
- [21] Kenneth O Stanley and Risto Miikkulainen. A taxonomy for artificial embryogeny. *Artificial Life*, 9(2):93–130, 2003.
- [22] Kenneth O Stanley. Compositional pattern producing networks: A novel abstraction of development. *Genetic Programming and Evolvable Machines*, 8(2):131–162, 2007.
- [23] Jeff Clune, Benjamin E Beckmann, Charles Ofria, and Robert T Pennock. Evolving coordinated quadruped gaits with the hyperneat generative encoding. In *Proceedings of the IEEE Congress on Evolutionary Computation*, pages 2764–2771, Trondheim, Norway, May 2009.

- [24] Joost Huizinga, Jeff Clune, and Jean-Baptiste Mouret. Evolving neural networks that are both modular and regular: Hyperneat plus the connection cost technique. In *Proceedings of the Annual Conference on Genetic and Evolutionary Computation*, pages 697–704, Vancouver, BC, Canada, July 2014.
- [25] Jacob Schrum, Joel Lehman, and Sebastian Risi. Automatic evolution of multimodal behavior with multi-brain hyperneat. In *Proceedings of the 2016 on Genetic and Evolutionary Computation Conference Companion*, pages 21–22, Denver, Colorado, USA, July 2016.
- [26] Kembra L Howdeshell. A model of the development of the brain as a construct of the thyroid system. *Environmental Health Perspectives*, 110:337, 2002.
- [27] Sh A Bayer, J Altman, RJ Russo, and Xin Zhang. Timetables of neurogenesis in the human brain based on experimentally determined patterns in the rat. *Neurotoxicology*, 14(1):83, 1993.
- [28] Gregory Z Tau and Bradley S Peterson. Normal development of brain circuits. *Neuropsychopharmacology*, 35(1):147–168, 2010.
- [29] Chris Eliasmith. *How to build a brain: A neural architecture for biological cognition*. Oxford University Press, 2013.
- [30] Jacob Schrum, Joel Lehman, and Sebastian Risi. Using indirect encoding of multiple brains to produce multimodal behavior. *CoRR*, abs/1604.07806, 2016.
- [31] John Rubenstein and Pasko Rakic. *Patterning and cell type specification in the developing CNS and PNS: Comprehensive developmental neuroscience*, volume 1. Academic Press, 2013.
- [32] Mark Bear, Barry Connors, and Mike Paradiso. *Neuroscience: Exploring the brain*. Wolters Kluwer, 4th edition, 2016.

- [33] Catherine Belzung and Peter Wigmore. *Neurogenesis and neural plasticity*. Springer, 2013.
- [34] Alexander D Rast, Francesco Galluppi, Xin Jin, and SB Furber. The leaky integrate-and-fire neuron: A platform for synaptic model exploration on the spinnaker chip. In *Proceedings of the IEEE International Joint Conference on Neural Networks (IJCNN)*, pages 1–8, Barcelona, Spain, July 2010.
- [35] Carlos Aramburo De la Hoz, Edward G Jones, and Jorge A Larriva Sahd. *From development to degeneration and regeneration of the nervous system*. Oxford University Press, 2008.
- [36] Maurizio Mattia and Paolo Del Giudice. Efficient event-driven simulation of large networks of spiking neurons and dynamical synapses. *Neural Computation*, 12:2305–2329, 2000.
- [37] Arnaud Delorme and Simon J Thorpe. Spikenet: an event-driven simulation package for modelling large networks of spiking neurons. *Network: Computation in Neural Systems*, 14(4):613–627, 2003.
- [38] Olivier Rochel and Dominique Martinez. An event-driven framework for the simulation of networks of spiking neurons. In *Proceedings of 11th European Symposium On Artificial Neural Networks (ESANN)*, Bruges, Belgium, April 2003.
- [39] Eduardo Ros, Richard Carrillo, Eva M. Ortigosa, Boris Barbour, and Rodrigo Agís. Event-driven simulation scheme for spiking neural networks using lookup tables to characterize neuronal dynamics. *Neural Computation*, 18(12):2959–2993, 2006.
- [40] Nikola K. Kasabov. *Time-Space, Spiking Neural Networks and Brain-Inspired Artificial Intelligence*. Springer, 2019.

- [41] Wulfram Gerstner and Werner M Kistler. *Spiking neuron models: Single neurons, populations, plasticity*. Cambridge university press, 2002.
- [42] Donald Olding Hebb. *The organization of behavior: A neuropsychological theory*. Psychology Press, 2005.
- [43] Jurgen Schmidhuber. Multi-column deep neural networks for image classification. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3642–3649, Providence, Rhode Island, USA, June 2012.
- [44] Fabrizio Gabbiani and W Metzner. Encoding and processing of sensory information in neuronal spike trains. *Journal of Experimental Biology*, 202(10):1267–1279, 1999.

APPENDICES

APPENDIX A

SAMPLE CONNECTOME

```
1  "2019-03-10_22:23:10_123834_N79ZWZ_N": {
2      "neighbors": {},
3      "event_id": {},
4      "membrane_potential": 0,
5      "cumulative_fire_count": 0,
6      "cumulative_fire_count_inst": 0,
7      "cumulative_intake_total": 0,
8      "cumulative_intake_count": 0,
9      "consecutive_fire_cnt": 0,
10     "snooze_till_burst_num": 0,
11     "last_burst_num": 0,
12     "activity_history": [],
13     "location": [
14         30,
15         23,
16         4
17     ],
18     "block": [
19         29,
20         22,
21         0
22     ],
23     "status": "Passive",
```

```

24     "last_membrane_potential_reset_time": "2019-03-10
      ↪ 22:23:10.123870",
25     "last_membrane_potential_reset_burst": 0,
26     "firing_pattern_id": "",
27     "activation_function_id": "",
28     "depolarization_threshold": 1.298473376883852,
29     "firing_threshold": 3.433882933618276
30 },
31 "2019-03-10_22:23:10_123879_WIDOPK_N": {
32     "neighbors": {
33         "2019-03-10_22:23:10_881769_1Y20ND_N": {
34             "cortical_area": "vision_v2",
35             "postsynaptic_current": 20
36         },
37         "2019-03-10_22:23:10_893846_IONUPX_N": {
38             "cortical_area": "vision_v2",
39             "postsynaptic_current": 20
40         }
41     },
42     "event_id": {},
43     "membrane_potential": 0,
44     "cumulative_fire_count": 0,
45     "cumulative_fire_count_inst": 0,
46     "cumulative_intake_total": 0,
47     "cumulative_intake_count": 0,
48     "consecutive_fire_cnt": 0,
49     "snooze_till_burst_num": 0,

```

```

50     "last_burst_num": 0,
51     "activity_history": [],
52     "location": [
53         2,
54         8,
55         1
56     ],
57     "block": [
58         1,
59         7,
60         0
61     ],
62     "status": "Passive",
63     "last_membrane_potential_reset_time": "2019-03-10
        ↪ 22:23:10.123898",
64     "last_membrane_potential_reset_burst": 0,
65     "firing_pattern_id": "",
66     "activation_function_id": "",
67     "depolarization_threshold": 1.298473376883852,
68     "firing_threshold": 3.433882933618276
69 },
70 "2019-03-10_22:23:10_123904_HYZ8HK_N": {
71     "neighbors": {
72         "2019-03-10_22:23:10_706899_QZZQWR_N": {
73             "cortical_area": "vision_v2",
74             "postsynaptic_current": 20
75         }

```

```

76     },
77     "event_id": {},
78     "membrane_potential": 0,
79     "cumulative_fire_count": 0,
80     "cumulative_fire_count_inst": 0,
81     "cumulative_intake_total": 0,
82     "cumulative_intake_count": 0,
83     "consecutive_fire_cnt": 0,
84     "snooze_till_burst_num": 0,
85     "last_burst_num": 0,
86     "activity_history": [],
87     "location": [
88         7,
89         8,
90         3
91     ],
92     "block": [
93         6,
94         7,
95         0
96     ],
97     "status": "Passive",
98     "last_membrane_potential_reset_time": "2019-03-10
    ↪ 22:23:10.123922",
99     "last_membrane_potential_reset_burst": 0,
100    "firing_pattern_id": "",
101    "activation_function_id": "",

```

```

102     "depolarization_threshold": 1.298473376883852,
103     "firing_threshold": 3.433882933618276
104 },
105 "2019-03-10_22:23:10_123928_VFQWS3_N": {
106     "neighbors": {
107         "2019-03-10_22:23:10_724294_4Y4KP3_N": {
108             "cortical_area": "vision_v2",
109             "postsynaptic_current": 20
110         },
111         "2019-03-10_22:23:10_822911_RCZZ8S_N": {
112             "cortical_area": "vision_v2",
113             "postsynaptic_current": 20
114         }
115     },
116     "event_id": {},
117     "membrane_potential": 0,
118     "cumulative_fire_count": 0,
119     "cumulative_fire_count_inst": 0,
120     "cumulative_intake_total": 0,
121     "cumulative_intake_count": 0,
122     "consecutive_fire_cnt": 0,
123     "snooze_till_burst_num": 0,
124     "last_burst_num": 0,
125     "activity_history": [],
126     "location": [
127         13,
128         12,

```

```

129         0
130     ],
131     "block": [
132         12,
133         11,
134         0
135     ],
136     "status": "Passive",
137     "last_membrane_potential_reset_time": "2019-03-10
        ↪ 22:23:10.123955",
138     "last_membrane_potential_reset_burst": 0,
139     "firing_pattern_id": "",
140     "activation_function_id": "",
141     "depolarization_threshold": 1.298473376883852,
142     "firing_threshold": 3.433882933618276
143 },
144 "2019-03-10_22:23:10_123962_5QZPK5_N": {
145     "neighbors": {
146         "2019-03-10_22:23:10_676868_040HPA_N": {
147             "cortical_area": "vision_v2",
148             "postsynaptic_current": 20
149         },
150         "2019-03-10_22:23:10_688269_ELCZUP_N": {
151             "cortical_area": "vision_v2",
152             "postsynaptic_current": 20
153         },
154         "2019-03-10_22:23:10_964638_0A8BBS_N": {

```

```

155         "cortical_area": "vision_v2",
156         "postsynaptic_current": 20
157     },
158     "2019-03-10_22:23:11_008964_G0S1Q9_N": {
159         "cortical_area": "vision_v2",
160         "postsynaptic_current": 20
161     }
162 },
163 "event_id": {},
164 "membrane_potential": 0,
165 "cumulative_fire_count": 0,
166 "cumulative_fire_count_inst": 0,
167 "cumulative_intake_total": 0,
168 "cumulative_intake_count": 0,
169 "consecutive_fire_cnt": 0,
170 "snooze_till_burst_num": 0,
171 "last_burst_num": 0,
172 "activity_history": [],
173 "location": [
174     28,
175     19,
176     5
177 ],
178 "block": [
179     27,
180     18,
181     0

```

```
182     ],
183     "status": "Passive",
184     "last_membrane_potential_reset_time": "2019-03-10
        ↪ 22:23:10.123980",
185     "last_membrane_potential_reset_burst": 0,
186     "firing_pattern_id": "",
187     "activation_function_id": "",
188     "depolarization_threshold": 1.298473376883852,
189     "firing_threshold": 3.433882933618276
190 }
```


APPENDIX B

SAMPLE GENOME

1	{	69	-1,
2	"neighbor_locator_rule" : {	70	1,
3	"rule_0" : {	71	-1
4	"param_1" : 0,	72	]
5	"param_2" : 0	73	],
6	},	74	" " : [
7	"rule_1" : {	75	[
8	"param_1" : 1,	76	0,
9	"param_2" : 1	77	0,
10	},	78	0
11	"rule_2" : {	79	],
12	"param_1" : 5,	80	[
13	"param_2" : 5,	81	0,
14	"param_3" : 10	82	0,
15	},	83	0
16	"rule_3" : {	84	],
17	"param_1" : 0,	85	[
18	"param_2" : 0	86	0,
19	},	87	0,
20	"rule_4" : {	88	0
21	"param_1" : 25,	89	]
22	"param_2" : 25	90	],
23	},	91	"o" : [
24	"rule_5" : {	92	[
25	"param_1" : 700,	93	-1,
26	"param_2" : 700,	94	-1,
27	"param_3" : 700,	95	-1
28	"param_4" : 700,	96	],
29	"param_5" : 700,	97	[
30	"param_6" : 700,	98	-1,
31	"param_7" : 700	99	4,
32	},	100	-1
33	"rule_6" : {	101	],
34	"param_1" : 1,	102	[
35	"param_2" : 1	103	-1,
36	}	104	-1,
37	},	105	-1
38	"IPU_vision_filters" : {	106	]
39	"3" : {	107	],
40	"-" : [	108	"/" : [
41	[	109	[
42	-1,	110	-1,
43	-1,	111	-1,
44	-1	112	1
45	],	113	],
46	[	114	[
47	1,	115	-1,
48	1,	116	1,
49	1	117	-1
50	],	118	],
51	[	119	[
52	-1,	120	1,
53	-1,	121	-1,
54	-1	122	-1
55	]	123	]
56	],	124	],
57	" " : [	125	"\\" : [
58	[	126	[
59	-1,	127	1,
60	1,	128	-1,
61	-1	129	-1
62	],	130	],
63	[	131	[
64	-1,	132	-1,
65	1,	133	1,
66	-1	134	-1
67	],	135	],
68	[	136	[

137	-1,	205	0,
138	-1,	206	0,
139	1	207	0,
140	]	208	0
141	]	209	],
142	},	210	[
143	"5" : {	211	0,
144	"_" : [	212	0,
145	[	213	0,
146	-1,	214	0,
147	-1,	215	0
148	-1,	216	]
149	-1,	217	],
150	-1	218	" " : [
151	],	219	[
152	[	220	-1,
153	-1,	221	-1,
154	-1,	222	1,
155	-1,	223	-1,
156	-1,	224	-1
157	-1	225	],
158	],	226	[
159	[	227	-1,
160	1,	228	-1,
161	1,	229	1,
162	1,	230	-1,
163	1,	231	-1
164	1	232	],
165	],	233	[
166	[	234	-1,
167	-1,	235	-1,
168	-1,	236	1,
169	-1,	237	-1,
170	-1,	238	-1
171	-1	239	],
172	],	240	[
173	[	241	-1,
174	-1,	242	-1,
175	-1,	243	1,
176	-1,	244	-1,
177	-1,	245	-1
178	-1	246	],
179	]	247	[
180	],	248	-1,
181	" " : [	249	-1,
182	[	250	1,
183	0,	251	-1,
184	0,	252	-1
185	0,	253	]
186	0,	254	],
187	0	255	"o" : [
188	],	256	[
189	[	257	-1,
190	0,	258	-1,
191	0,	259	-1,
192	0,	260	-1,
193	0,	261	-1
194	0	262	],
195	],	263	[
196	[	264	-1,
197	0,	265	-1,
198	0,	266	-1,
199	0,	267	-1,
200	0,	268	-1
201	0	269	],
202	],	270	[
203	[	271	-1,
204	0,	272	-1,

273	-1,	341	-1,
274	-1,	342	-1
275	-1	343	],
276	],	344	[
277	[	345	-1,
278	-1,	346	-1,
279	-1,	347	1,
280	-1,	348	-1,
281	-1,	349	-1
282	-1	350	],
283	],	351	[
284	[	352	-1,
285	-1,	353	-1,
286	-1,	354	-1,
287	-1,	355	1,
288	-1,	356	-1
289	-1	357	],
290	]	358	[
291	],	359	-1,
292	"/" : [	360	-1,
293	[	361	-1,
294	-1,	362	-1,
295	-1,	363	1
296	-1,	364	]
297	-1,	365	]
298	1	366	},
299	],	367	"7" : {
300	[	368	"_" : [
301	-1,	369	[
302	-1,	370	-1,
303	-1,	371	-1,
304	1,	372	-1,
305	-1	373	-1,
306	],	374	-1,
307	[	375	-1,
308	-1,	376	-1
309	-1,	377	],
310	1,	378	[
311	-1,	379	-1,
312	-1	380	-1,
313	],	381	-1,
314	[	382	-1,
315	-1,	383	-1,
316	1,	384	-1,
317	-1,	385	-1
318	-1,	386	],
319	-1	387	[
320	],	388	-1,
321	[	389	-1,
322	1,	390	-1,
323	-1,	391	-1,
324	-1,	392	-1,
325	-1,	393	-1,
326	-1	394	-1
327	]	395	],
328	],	396	[
329	"\\" : [	397	1,
330	[	398	1,
331	1,	399	1,
332	-1,	400	1,
333	-1,	401	1,
334	-1,	402	1,
335	-1	403	1
336	],	404	],
337	[	405	[
338	-1,	406	-1,
339	1,	407	-1,
340	-1,	408	-1,

409	-1,	477	0
410	-1,	478	],
411	-1,	479	[
412	-1	480	0,
413	],	481	0,
414	[	482	0,
415	-1,	483	0,
416	-1,	484	0,
417	-1,	485	0,
418	-1,	486	0
419	-1,	487	],
420	-1,	488	[
421	-1	489	0,
422	],	490	0,
423	[	491	0,
424	-1,	492	0,
425	-1,	493	0,
426	-1,	494	0,
427	-1,	495	0
428	-1,	496	]
429	-1,	497	],
430	-1	498	"I" : [
431	]	499	[
432	],	500	-1,
433	" " : [	501	-1,
434	[	502	-1,
435	0,	503	1,
436	0,	504	-1,
437	0,	505	-1,
438	0,	506	-1
439	0,	507	],
440	0,	508	[
441	0	509	-1,
442	],	510	-1,
443	[	511	-1,
444	0,	512	1,
445	0,	513	-1,
446	0,	514	-1,
447	0,	515	-1
448	0,	516	],
449	0,	517	[
450	0	518	-1,
451	],	519	-1,
452	[	520	-1,
453	0,	521	1,
454	0,	522	-1,
455	0,	523	-1,
456	0,	524	-1
457	0,	525	],
458	0,	526	[
459	0	527	-1,
460	],	528	-1,
461	[	529	-1,
462	0,	530	1,
463	0,	531	-1,
464	0,	532	-1,
465	0,	533	-1
466	0,	534	],
467	0,	535	[
468	0	536	-1,
469	],	537	-1,
470	[	538	-1,
471	0,	539	1,
472	0,	540	-1,
473	0,	541	-1,
474	0,	542	-1
475	0,	543	],
476	0,	544	[

545	-1,	613	1,
546	-1,	614	-1,
547	-1,	615	-1,
548	1,	616	-1
549	-1,	617	],
550	-1,	618	[
551	-1	619	-1,
552	],	620	-1,
553	[	621	-1,
554	-1,	622	-1,
555	-1,	623	-1,
556	-1,	624	-1,
557	1,	625	-1
558	-1,	626	]
559	-1,	627	],
560	-1	628	"o" : [
561	]	629	[
562	],	630	-1,
563	"i" : [	631	-1,
564	[	632	-1,
565	-1,	633	-1,
566	-1,	634	-1,
567	-1,	635	-1,
568	-1,	636	-1
569	-1,	637	],
570	-1,	638	[
571	-1	639	-1,
572	],	640	-1,
573	[	641	-1,
574	-1,	642	-1,
575	-1,	643	-1,
576	-1,	644	-1,
577	1,	645	-1
578	-1,	646	],
579	-1,	647	[
580	-1	648	-1,
581	],	649	-1,
582	[	650	-1,
583	-1,	651	-1,
584	-1,	652	-1,
585	-1,	653	-1,
586	1,	654	-1
587	-1,	655	],
588	-1,	656	[
589	-1	657	-1,
590	],	658	-1,
591	[	659	-1,
592	-1,	660	-1,
593	-1,	661	-1,
594	-1,	662	-1,
595	1,	663	-1
596	-1,	664	],
597	-1,	665	[
598	-1	666	-1,
599	],	667	-1,
600	[	668	-1,
601	-1,	669	-1,
602	-1,	670	-1,
603	-1,	671	-1,
604	1,	672	-1
605	-1,	673	],
606	-1,	674	[
607	-1	675	-1,
608	],	676	-1,
609	[	677	-1,
610	-1,	678	-1,
611	-1,	679	-1,
612	-1,	680	-1,

681	-1	749	1,
682	],	750	-1,
683	[	751	-1,
684	-1,	752	-1,
685	-1,	753	-1,
686	-1,	754	-1,
687	-1,	755	-1
688	-1,	756	]
689	-1,	757	],
690	-1	758	"\\": [
691	]	759	[
692	],	760	1,
693	"/": [	761	-1,
694	[	762	-1,
695	-1,	763	-1,
696	-1,	764	-1,
697	-1,	765	-1,
698	-1,	766	-1
699	-1,	767	],
700	-1,	768	[
701	1	769	-1,
702	],	770	1,
703	[	771	-1,
704	-1,	772	-1,
705	-1,	773	-1,
706	-1,	774	-1,
707	-1,	775	-1
708	-1,	776	],
709	1,	777	[
710	-1	778	-1,
711	],	779	-1,
712	[	780	1,
713	-1,	781	-1,
714	-1,	782	-1,
715	-1,	783	-1,
716	-1,	784	-1
717	1,	785	],
718	-1,	786	[
719	-1	787	-1,
720	],	788	-1,
721	[	789	-1,
722	-1,	790	1,
723	-1,	791	-1,
724	-1,	792	-1,
725	1,	793	-1
726	-1,	794	],
727	-1,	795	[
728	-1	796	-1,
729	],	797	-1,
730	[	798	-1,
731	-1,	799	-1,
732	-1,	800	1,
733	1,	801	-1,
734	-1,	802	-1
735	-1,	803	],
736	-1,	804	[
737	-1	805	-1,
738	],	806	-1,
739	[	807	-1,
740	-1,	808	-1,
741	1,	809	-1,
742	-1,	810	1,
743	-1,	811	-1
744	-1,	812	],
745	-1,	813	[
746	-1	814	-1,
747	],	815	-1,
748	[	816	-1,

```

817         -1,
818         -1,
819         -1,
820         1
821     ]
822 ]
823 }
824 },
825 "location_tolerance" : 2,
826 "image_color_intensity_tolerance" : 250,
827 "max_burst_count" : 3,
828 "evolution_burst_count" : 100,
829 "performance_stats" : {
830     "mnist_view_cnt" : 0,
831     "mnist_correct_detection_cnt" : 0
832 },
833 "blueprint" : {
834     "vision_v1-1" : {
835         "growth_path" : "",
836         "direction_sensitivity" : "/",
837         "group_id" : "vision",
838         "sub_group_id" : "vision_v1",
839         "plot_index" : 1,
840         "layer_index" : 1,
841         "total_layer_count" : 7,
842         "orientation_selectivity_pattern" :
            ↳ "",
843         "location" : "",
844         "kernel_size" : 3,
845         "cortical_neuron_count" : 1000,
846         "location_generation_type" : "random
            ↳ ",
847         "synapse_attractivity" : 100,
848         "init_synapse_needed" : false,
849         "postsynaptic_current" : 20,
850         "plasticity_constant" : 1,
851         "postsynaptic_current_max" : 1000,
852         "neighbor_locator_rule_id" : "rule_1
            ↳ ",
853         "neighbor_locator_rule_param_id" : "
            ↳ param_1",
854         "cortical_mapping_dst" : {
855             "vision_v2" : {
856                 "neighbor_locator_rule_id" : "
                    ↳ rule_5",
857                 "
                    ↳ neighbor_locator_rule_param_id
                    ↳ " : "param_1"
            }
        },
858     },
859 },
860 "neuron_params" : {
861     "activation_function_id" : "",
862     "depolarization_threshold" :
        ↳ 1.76435,
863     "orientation_selectivity_id" : "",
864     "firing_threshold" : 3.9567,
865     "firing_pattern_id" : "",
866     "refractory_period" : 0,
867     "axon_avg_length" : "",
868     "leak_coefficient" : 10,
869     "axon_avg_connections" : "",
870     "axon_orientation_function" : "",
871     "consecutive_fire_cnt_max" : 1,
872     "snooze_length" : 1.80804,
873     "block_boundaries" : [28,28,1],
874     "geometric_boundaries" : {
875         "x" : [
876             0,
877             28
878         ],
879         "y" : [
880             0,
881             28
882         ],
883         "z" : [
884             0,
885             10
886         ]
887     }
888 },
889 },
890 "vision_v1-2" : {
891     "growth_path" : "",
892     "group_id" : "vision",
893     "sub_group_id" : "vision_v1",
894     "plot_index" : 4,
895     "layer_index" : 2,
896     "total_layer_count" : 7,
897     "direction_sensitivity" : "\\ ",
898     "orientation_selectivity_pattern" :
        ↳ "",
899     "location" : "",
900     "kernel_size" : 3,
901     "cortical_neuron_count" : 1000,
902     "location_generation_type" : "random
        ↳ ",
903     "synapse_attractivity" : 100,
904     "init_synapse_needed" : false,
905     "postsynaptic_current" : 20,
906     "plasticity_constant" : 1,
907     "postsynaptic_current_max" : 1000,
908     "neighbor_locator_rule_id" : "rule_1
        ↳ ",
909     "neighbor_locator_rule_param_id" : "
        ↳ param_1",
910     "cortical_mapping_dst" : {
911         "vision_v2" : {
912             "neighbor_locator_rule_id" : "
                ↳ rule_5",
913             "
                ↳ neighbor_locator_rule_param_id
                ↳ " : "param_2"
        }
914     },
915 },
916 "neuron_params" : {
917     "activation_function_id" : "",
918     "depolarization_threshold" :
        ↳ 1.76435,
919     "orientation_selectivity_id" : "",
920     "firing_threshold" : 3.9567,
921     "firing_pattern_id" : "",
922     "refractory_period" : 0,
923     "axon_avg_length" : "",
924     "leak_coefficient" : 10,
925     "axon_avg_connections" : "",
926     "axon_orientation_function" : "",
927     "consecutive_fire_cnt_max" : 1,
928     "snooze_length" : 1.80804,
929     "block_boundaries" : [28,28,1],
930     "geometric_boundaries" : {
931         "x" : [
932             0,
933             28
934         ],
935         "y" : [
936             0,

```


937	28	997	10
938	],	998	]
939	"z" : [	999	]
940	0,	1000	}
941	10	1001	},
942	]	1002	"vision_v1-4" : {
943	}	1003	"growth_path" : "",
944	}	1004	"group_id" : "vision",
945	},	1005	"sub_group_id" : "vision_v1",
946	"vision_v1-3" : {	1006	"plot_index" : 10,
947	"growth_path" : "",	1007	"layer_index" : 4,
948	"group_id" : "vision",	1008	"total_layer_count" : 7,
949	"sub_group_id" : "vision_v1",	1009	"direction_sensitivity" : "-",
950	"plot_index" : 7,	1010	"orientation_selectivity_pattern" :
951	"layer_index" : 3,		↪ "",
952	"total_layer_count" : 7,	1011	"location" : "",
953	"direction_sensitivity" : " ",	1012	"kernel_size" : 3,
954	"orientation_selectivity_pattern" :	1013	"cortical_neuron_count" : 1000,
	↪ "",	1014	"location_generation_type" : "random
955	"location" : "",		↪ ",
956	"kernel_size" : 3,	1015	"synapse_attractivity" : 100,
957	"cortical_neuron_count" : 1000,	1016	"init_synapse_needed" : false,
958	"location_generation_type" : "random	1017	"postsynaptic_current" : 20,
	↪ ",	1018	"plasticity_constant" : 1,
959	"synapse_attractivity" : 100,	1019	"postsynaptic_current_max" : 1100,
960	"init_synapse_needed" : false,	1020	"neighbor_locator_rule_id" : "rule_1
961	"postsynaptic_current" : 20,		↪ ",
962	"plasticity_constant" : 1,	1021	"neighbor_locator_rule_param_id" : "
963	"postsynaptic_current_max" : 1000,		↪ param_1",
964	"neighbor_locator_rule_id" : "rule_1	1022	"cortical_mapping_dst" : {
	↪ ",	1023	"vision_v2" : {
965	"neighbor_locator_rule_param_id" : "	1024	"neighbor_locator_rule_id" : "
	↪ param_1",		↪ rule_5",
966	"cortical_mapping_dst" : {	1025	"
967	"vision_v2" : {		↪ neighbor_locator_rule_param_id
968	"neighbor_locator_rule_id" : "		↪ " : "param_4"
	↪ rule_5",	1026	}
969	"	1027	},
	↪ neighbor_locator_rule_param_id	1028	"neuron_params" : {
	↪ " : "param_3"	1029	"activation_function_id" : "",
	}	1030	"depolarization_threshold" :
970	},		↪ 1.76435,
971	"neuron_params" : {	1031	"orientation_selectivity_id" : "",
972	"activation_function_id" : "",	1032	"firing_threshold" : 3.9567,
973	"depolarization_threshold" :	1033	"firing_pattern_id" : "",
974	↪ 1.76435,	1034	"refractory_period" : 0,
975	"orientation_selectivity_id" : "",	1035	"axon_avg_length" : "",
976	"firing_threshold" : 3.9567,	1036	"leak_coefficient" : 10,
977	"firing_pattern_id" : "",	1037	"axon_avg_connections" : "",
978	"refractory_period" : 0,	1038	"axon_orientation function" : "",
979	"axon_avg_length" : "",	1039	"consecutive_fire_cnt_max" : 1,
980	"leak_coefficient" : 10,	1040	"snooze_length" : 1.80804,
981	"axon_avg_connections" : "",	1041	"block_boundaries" : [28,28,1],
982	"axon_orientation function" : "",	1042	"geometric_boundaries" : {
983	"consecutive_fire_cnt_max" : 1,	1043	"x" : [
984	"snooze_length" : 1.80804,	1044	0,
985	"block_boundaries" : [28,28,1],	1045	28
986	"geometric_boundaries" : {	1046	],
987	"x" : [	1047	"y" : [
988	0,	1048	0,
989	28	1049	28
990	],	1050	],
991	"y" : [	1051	"z" : [
992	0,	1052	0,
993	28	1053	10
994	],	1054	]
995	"z" : [	1055	}
996	0,	1056	}

1057	},	1117	"sub_group_id" : "vision_v1",
1058	"vision_v1-5" : {	1118	"plot_index" : 16,
1059	"growth_path" : "",	1119	"layer_index" : 6,
1060	"group_id" : "vision",	1120	"total_layer_count" : 7,
1061	"sub_group_id" : "vision_v1",	1121	"direction_sensitivity" : "o",
1062	"plot_index" : 13,	1122	"orientation_selectivity_pattern" :
1063	"layer_index" : 5,		↪ "",
1064	"total_layer_count" : 7,	1123	"location" : "",
1065	"direction_sensitivity" : "I",	1124	"kernel_size" : 3,
1066	"orientation_selectivity_pattern" :	1125	"cortical_neuron_count" : 1000,
	↪ "",	1126	"location_generation_type" : "random
1067	"location" : "",		↪ ",
1068	"kernel_size" : 3,	1127	"synapse_attractivity" : 100,
1069	"cortical_neuron_count" : 1000,	1128	"init_synapse_needed" : false,
1070	"location_generation_type" : "random	1129	"postsynaptic_current" : 20,
	↪ ",	1130	"plasticity_constant" : 1,
1071	"synapse_attractivity" : 100,	1131	"postsynaptic_current_max" : 1000,
1072	"init_synapse_needed" : false,	1132	"neighbor_locator_rule_id" : "rule_1
1073	"postsynaptic_current" : 20,		↪ ",
1074	"plasticity_constant" : 1,	1133	"neighbor_locator_rule_param_id" : "
1075	"postsynaptic_current_max" : 1000,		↪ param_1",
1076	"neighbor_locator_rule_id" : "rule_1	1134	"cortical_mapping_dst" : {
	↪ ",	1135	"vision_v2" : {
1077	"neighbor_locator_rule_param_id" : "	1136	"neighbor_locator_rule_id" : "
	↪ param_1",		↪ rule_5",
1078	"cortical_mapping_dst" : {	1137	"
1079	"vision_v2" : {		↪ neighbor_locator_rule_param_id
1080	"neighbor_locator_rule_id" : "		↪ " : "param_6"
	↪ rule_5",		}
1081	"	1138	
	↪ neighbor_locator_rule_param_id	1139	},
	↪ " : "param_5"	1140	"neuron_params" : {
	}	1141	"activation_function_id" : "",
1082	},	1142	"depolarization_threshold" :
1083	"neuron_params" : {		↪ 1.76435,
1084	"activation_function_id" : "",	1143	"orientation_selectivity_id" : "",
1085	"depolarization_threshold" :	1144	"firing_threshold" : 3.9567,
1086	↪ 1.76435,	1145	"firing_pattern_id" : "",
1087	"orientation_selectivity_id" : "",	1146	"refractory_period" : 0,
1088	"firing_threshold" : 3.9567,	1147	"axon_avg_length" : "",
1089	"firing_pattern_id" : "",	1148	"leak_coefficient" : 10,
1090	"refractory_period" : 0,	1149	"axon_avg_connections" : "",
1091	"axon_avg_length" : "",	1150	"axon_orientation_function" : "",
1092	"leak_coefficient" : 10,	1151	"consecutive_fire_cnt_max" : 1,
1093	"axon_avg_connections" : "",	1152	"snooze_length" : 1.80804,
1094	"axon_orientation_function" : "",	1153	"block_boundaries" : [28,28,1],
1095	"consecutive_fire_cnt_max" : 1,	1154	"geometric_boundaries" : {
1096	"snooze_length" : 1.80804,	1155	"x" : [
1097	"block_boundaries" : [28,28,1],	1156	0,
1098	"geometric_boundaries" : {	1157	28
1099	"x" : [	1158	],
1100	0,	1159	"y" : [
1101	28	1160	0,
1102	],	1161	28
1103	"y" : [	1162	],
1104	0,	1163	"z" : [
1105	28	1164	0,
1106	],	1165	10
1107	"z" : [	1166	]
1108	0,	1167	}
1109	10	1168	}
1110	]	1169	},
1111	}	1170	"vision_v1-7" : {
1112	}	1171	"growth_path" : "",
1113	},	1172	"group_id" : "vision",
1114	"vision_v1-6" : {	1173	"sub_group_id" : "vision_v1",
1115	"growth_path" : "",	1174	"plot_index" : 19,
1116	"group_id" : "vision",	1175	"layer_index" : 7,
		1176	"total_layer_count" : 7,

```

1177     "direction_sensitivity" : " ",      1235
1178     "orientation_selectivity_pattern" :
1179         ↪ " ",                          1236
1180     "location" : "",                  1237
1181     "kernel_size" : 3,                1238
1182     "cortical_neuron_count" : 1000,   1239
1183     "location_generation_type" : "random 1240
1184         ↪ " ",                          1241
1185     "synapse_attractivity" : 100,
1186     "init_synapse_needed" : false,    1242
1187     "postsynaptic_current" : 20,
1188     "plasticity_constant" : 1,        1243
1189     "postsynaptic_current_max" : 1000, 1244
1190     "neighbor_locator_rule_id" : "rule_1 1245
1191         ↪ " ",                          1246
1192     "neighbor_locator_rule_param_id" : " 1247
1193         ↪ param_1",                    1248
1194     "cortical_mapping_dst" : {        1249
1195         "vision_v2" : {                1250
1196             "neighbor_locator_rule_id" : " 1251
1197                 ↪ rule_5",            1252
1198                 ↪ neighbor_locator_rule_param_id 1253
1199                 ↪ " : "param_7"      1254
1200         },                            1255
1201     },                                1256
1202     "neuron_params" : {               1257
1203         "activation_function_id" : "",  1258
1204         "depolarization_threshold" :   1259
1205             ↪ 1.76435,                1260
1206         "orientation_selectivity_id" : "", 1261
1207         "firing_threshold" : 3.9567,    1262
1208         "firing_pattern_id" : "",       1263
1209         "refractory_period" : 0,        1264
1210         "axon_avg_length" : "",        1265
1211         "leak_coefficient" : 10,        1266
1212         "axon_avg_connections" : "",    1267
1213         "axon_orientation_function" : "", 1268
1214         "consecutive_fire_cnt_max" : 1,  1269
1215         "snore_length" : 1.80804,      1270
1216         "block_boundaries" : [28,28,1], 1271
1217         "geometric_boundaries" : {      1272
1218             "x" : [                    1273
1219                 0,                    1274
1220                 28                   1275
1221             ],                        1276
1222             "y" : [                    1277
1223                 0,                    1278
1224                 28                   1279
1225             ],                        1280
1226             "z" : [                    1281
1227                 0,                    1282
1228                 10                   1283
1229             ],                        1284
1230         },                            1285
1231     },                                1286
1232     "vision_v2" : {                   1287
1233         "growth_path" : "",            1288
1234         "group_id" : "vision",          1289
1235         "sub_group_id" : "vision_v2",   1290
1236         "plot_index" : 2,              1291
1237         "orientation_selectivity_pattern" :
1238             ↪ " ",                      1292
1239         "location" : "",               1293
1240         "kernel_size" : 3,
1241         "cortical_neuron_count" : 10000,

```

```

"location_generation_type" : "random
    ↪ " ,
"synapse_attractivity" : 100,
"init_synapse_needed" : false,
"postsynaptic_current" : 150,
"plasticity_constant" : 0.5,
"postsynaptic_current_max" : 200,
"neighbor_locator_rule_id" : "rule_1
    ↪ " ,
"neighbor_locator_rule_param_id" : "
    ↪ param_2",
"cortical_mapping_dst" : {
    "vision_IT" : {
        "neighbor_locator_rule_id" : "
            ↪ rule_6",
        "
            ↪ neighbor_locator_rule_param_id
            ↪ " : "param_1"
    }
},
"neuron_params" : {
    "activation_function_id" : "",
    "depolarization_threshold" :
        ↪ 1.76435,
    "firing_threshold" : 1.3189,
    "firing_pattern_id" : "",
    "refractory_period" : 0,
    "orientation_selectivity_id" : "",
    "axon_avg_length" : "",
    "leak_coefficient" : 5,
    "axon_avg_connections" : "",
    "axon_orientation_function" : "",
    "consecutive_fire_cnt_max" : 1,
    "snore_length" : 0,
    "block_boundaries" : [28,28,7],
    "geometric_boundaries" : {
        "x" : [
            0,
            70
        ],
        "y" : [
            0,
            70
        ],
        "z" : [
            0,
            70
        ]
    }
}
},
"vision_IT" : {
    "growth_path" : "",
    "group_id" : "vision",
    "sub_group_id" : "vision_IT",
    "plot_index" : 3,
    "orientation_selectivity_pattern" :
        ↪ " ",
    "location" : "",
    "kernel_size" : 7,
    "cortical_neuron_count" : 12000,
    "location_generation_type" : "random
        ↪ " ,
    "synapse_attractivity" : 80,
    "init_synapse_needed" : false,
    "postsynaptic_current" : 100,
    "plasticity_constant" : 0.5,
    "postsynaptic_current_max" : 300,

```

```

1294     "neighbor_locator_rule_id" : "rule_1" 1352
1295         ↪ ", 1353
1296     "neighbor_locator_rule_param_id" : " 1354
1297         ↪ param_2", 1355
1298     "cortical_mapping_dst" : { 1356
1299         "vision_memory" : { 1357
1300             "neighbor_locator_rule_id" : " 1358
1301                 ↪ rule_6", 1359
1302             " 1360
1303                 ↪ neighbor_locator_rule_param_id 1361
1304                 ↪ " : "param_2" 1362
1305             } 1363
1306         }, 1364
1307     "neuron_params" : { 1365
1308         "activation_function_id" : "", 1366
1309         "orientation_selectivity_id" : "", 1367
1310         "depolarization_threshold" : 1368
1311             ↪ 1.76435, 1369
1312         "firing_threshold" : 1, 1370
1313         "firing_pattern_id" : "", 1371
1314         "refractory_period" : 0, 1372
1315         "axon_avg_length" : "", 1373
1316         "leak_coefficient" : 5, 1374
1317         "axon_avg_connections" : "", 1375
1318         "axon_orientation_function" : "", 1376
1319         "consecutive_fire_cnt_max" : 1, 1377
1320         "snooze_length" : 1.00804, 1378
1321         "block_boundaries" : [28,28,1], 1379
1322         "geometric_boundaries" : { 1380
1323             "x" : [ 1381
1324                 0, 1382
1325                 70 1383
1326             ], 1384
1327             "y" : [ 1385
1328                 0, 1386
1329                 70 1387
1330             ], 1388
1331             "z" : [ 1389
1332                 0, 1390
1333                 70 1391
1334             ] 1392
1335         } 1393
1336     }, 1394
1337     "vision_memory" : { 1395
1338         "growth_path" : "", 1396
1339         "group_id" : "Memory", 1397
1340         "sub_group_id" : "vision", 1398
1341         "plot_index" : 1, 1399
1342         "orientation_selectivity_pattern" : 1400
1343             ↪ "", 1401
1344         "location" : "", 1402
1345         "kernel_size" : 7, 1403
1346         "cortical_neuron_count" : 10000, 1404
1347         "location_generation_type" : "random 1405
1348             ↪ ", 1406
1349         "synapse_attractivity" : 10, 1407
1350         "init_synapse_needed" : false, 1408
1351         "postsynaptic_current" : 0.9, 1409
1352         "plasticity_constant" : 1, 1410
1353         "postsynaptic_current_max" : 5, 1411
1354         "neighbor_locator_rule_id" : "rule_0 1412
1355             ↪ ", 1413
1356         "neighbor_locator_rule_param_id" : " 1414
1357             ↪ param_1", 1415
1358         "cortical_mapping_dst" : { 1416
1359             "utf8_memory" : { 1417
1360                 "neighbor_locator_rule_id" : " 1418
1361                     ↪ rule_3", 1419
1362                 " 1420
1363                     ↪ neighbor_locator_rule_param_id 1421
1364                     ↪ " : "param_2" 1422
1365                 } 1423
1366             }, 1424
1367         "neuron_params" : { 1425
1368             "activation_function_id" : "", 1426
1369             "orientation_selectivity_id" : "", 1427
1370             "depolarization_threshold" : 5, 1428
1371             "firing_threshold" : 1, 1429
1372             "firing_pattern_id" : "", 1430
1373             "refractory_period" : 0, 1431
1374             "axon_avg_length" : "", 1432
1375             "leak_coefficient" : 10, 1433
1376             "axon_avg_connections" : "", 1434
1377             "orientation_selectivity_id" : "", 1435
1378             "depolarization_threshold" : 5, 1436
1379             "firing_threshold" : 1, 1437
1380             "firing_pattern_id" : "", 1438
1381             "refractory_period" : 0, 1439
1382             "axon_avg_length" : "", 1440
1383             "leak_coefficient" : 10, 1441
1384             "axon_avg_connections" : "", 1442

```

```

1413         "axon_orientation function" : "", 1473
1414         "consecutive_fire_cnt_max" : 3, 1474
1415         "snooze_length" : 0, 1475
1416         "block_boundaries" : [1,1,300], 1476
1417         "geometric_boundaries" : { 1477
1418             "x" : [ 1478
1419                 0, 1479
1420                 1 1480
1421             ], 1481
1422             "y" : [ 1482
1423                 0, 1483
1424                 1 1484
1425             ], 1485
1426             "z" : [ 1486
1427                 0, 1487
1428                 300 1488
1429             ] 1489
1430         } 1490
1431     } 1491
1432 }, 1492
1433     "utf8_memory" : { 1493
1434         "growth_path" : "", 1494
1435         "group_id" : "Memory", 1495
1436         "sub_group_id" : "utf8", 1496
1437         "plot_index" : 2, 1497
1438         "orientation_selectivity_pattern" : 1498
1439             ↳ "", 1499
1440         "location" : "", 1500
1441         "kernel_size" : 7, 1501
1442         "cortical_neuron_count" : 300, 1502
1443         "location_generation_type" : " 1503
1444             ↳ sequential", 1504
1445         "synapse_attractivity" : 100, 1505
1446         "init_synapse_needed" : false, 1506
1447         "postsynaptic_current" : 11.2, 1507
1448         "plasticity_constant" : 1, 1508
1449         "postsynaptic_current_max" : 1, 1509
1450         "neighbor_locator_rule_id" : "rule_0 1510
1451             ↳ "", 1511
1452         "neighbor_locator_rule_param_id" : " 1512
1453             ↳ param_1", 1513
1454         "cortical_mapping_dst" : { 1514
1455             "utf8_out" : { 1515
1456                 "neighbor_locator_rule_id" : " 1516
1457                     ↳ rule_3", 1517
1458                 " 1518
1459                     ↳ neighbor_locator_rule_param_1 1519
1460                     ↳ " : param_2" 1520
1461             } 1521
1462         }, 1522
1463         "neuron_params" : { 1523
1464             "activation_function_id" : "", 1524
1465             "orientation_selectivity_id" : "", 1525
1466             "depolarization_threshold" : 20, 1526
1467             "firing_threshold" : 10, 1527
1468             "firing_pattern_id" : "", 1528
1469             "refractory_period" : 0, 1529
1470             "axon_avg_length" : "", 1530
1471             "leak_coefficient" : 10, 1531
1472             "axon_avg_connections" : "", 1532
1473             "axon_orientation function" : "", 1533
1474             "consecutive_fire_cnt_max" : 100000, 1534
1475             ↳ 100000, 1535
1476             "snooze_length" : 0,
1477             "block_boundaries" : [1,1,300],
1478             "geometric_boundaries" : {
1479                 "x" : [
1480                     0,
1481                     1
1482                 ],
1483                 "y" : [
1484                     0,
1485                     1
1486                 ],
1487                 "z" : [
1488                     0,
1489                     300
1490                 ]
1491             }
1492         }
1493     }
1494 },
1495     "utf8_out" : {
1496         "growth_path" : "",
1497         "group_id" : "OPU",
1498         "sub_group_id" : "OPU_utf8",
1499         "plot_index" : 1,
1500         "orientation_selectivity_pattern" :
1501             ↳ "",
1502         "location" : "",
1503         "kernel_size" : 7,
1504         "cortical_neuron_count" : 300,
1505         "location_generation_type" : "
1506             ↳ sequential",
1507         "synapse_attractivity" : 100,
1508         "init_synapse_needed" : false,
1509         "postsynaptic_current" : 0.51,
1510         "plasticity_constant" : 0.05,
1511         "postsynaptic_current_max" : 1,
1512         "neighbor_locator_rule_id" : "rule_0
1513             ↳ ",
1514         "neighbor_locator_rule_param_id" : "
1515             ↳ param_1",
1516         "cortical_mapping_dst" : {},
1517         "neuron_params" : {
1518             "activation_function_id" : "",
1519             "orientation_selectivity_id" : "",
1520             "depolarization_threshold" : 20,
1521             "firing_threshold" : 10,
1522             "firing_pattern_id" : "",
1523             "refractory_period" : 0,
1524             "axon_avg_length" : "",
1525             "leak_coefficient" : 50,
1526             "axon_avg_connections" : "",
1527             "axon_orientation function" : "",
1528             "consecutive_fire_cnt_max" : 1,
1529             "snooze_length" : 0,
1530             "block_boundaries" : [1,1,300],
1531             "geometric_boundaries" : {
1532                 "x" : [
1533                     0,
1534                     1
1535                 ],
1536                 "y" : [
1537                     0,
1538                     1
1539                 ],
1540                 "z" : [
1541                     0,
1542                     300
1543                 ]
1544             }
1545         }
1546     }
1547 },
1548     }
1549 }
1550 }

```